



群名称:机器视觉2026年春
群 号:737288720



第三讲：图像处理

授课人：王国泰

电子科技大学机械与电气工程学院

2026-3-10



知识回顾



• 相机内部参数与齐次坐标

原始坐标

$$\begin{aligned}x &= \frac{\phi_x u + \gamma v}{w} + \delta_x \\y &= \frac{\phi_y v}{w} + \delta_y\end{aligned}$$

齐次坐标

$$\begin{aligned}x &= \phi_x u + \gamma v + \delta_x w \\y &= \phi_y v + \delta_y w \\z &= w\end{aligned}$$

齐次坐标的矩阵形式

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \phi_x & \gamma & \delta_x \\ 0 & \phi_y & \delta_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

相机的内参可以表示为一个3x3的**矩阵** K
 K 为一个上三角矩阵

$$\mathbf{x} = K\mathbf{u}$$

图像中的坐标

三维空间中的坐标

知识回顾



- Intrinsic parameters 内部参数 (stored as intrinsic matrix)

$$\{\phi_x, \phi_y, \delta_x, \delta_y, \gamma\}$$

$$K = \begin{bmatrix} \phi_x & \gamma & \delta_x \\ 0 & \phi_y & \delta_y \\ 0 & 0 & 1 \end{bmatrix}$$

- Extrinsic parameters 外部参数

$$\{R, T\}$$

图像中的
二维点

$$\begin{bmatrix} wu \\ wv \\ w \end{bmatrix} = K \cdot \underbrace{[R \mid T]}_{\text{Projection Matrix } P} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

世界坐标系
中的三维点

投影矩阵

K: project (5 d.o.f.)

R: rotate (3 d.o.f.)

T: translate (3 d.o.f.)

Image processing basics

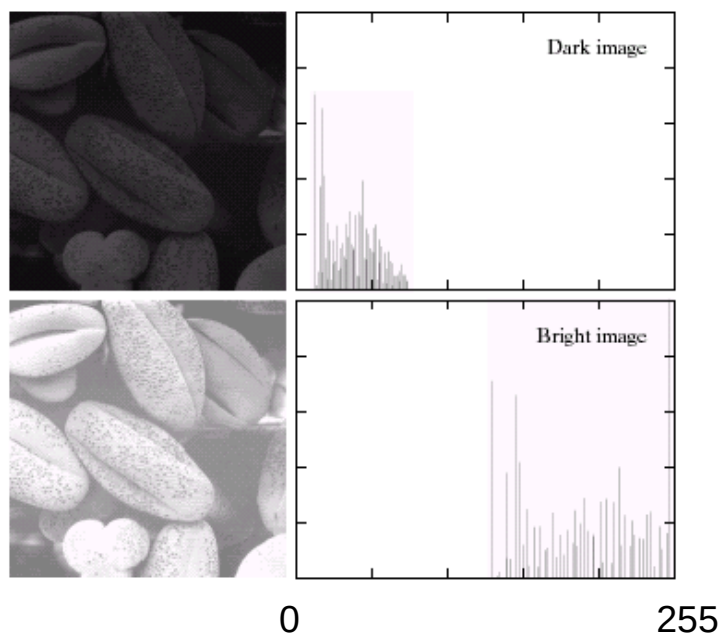


- Point processing 点处理
 - Gamma correction
 - Histogram equalization
 - Window-level correction
- Filtering 滤波
 - Mean, gradient
 - Convolution, cross-correlation
 - Gaussian and median filters
- Frequency analysis 频率分析

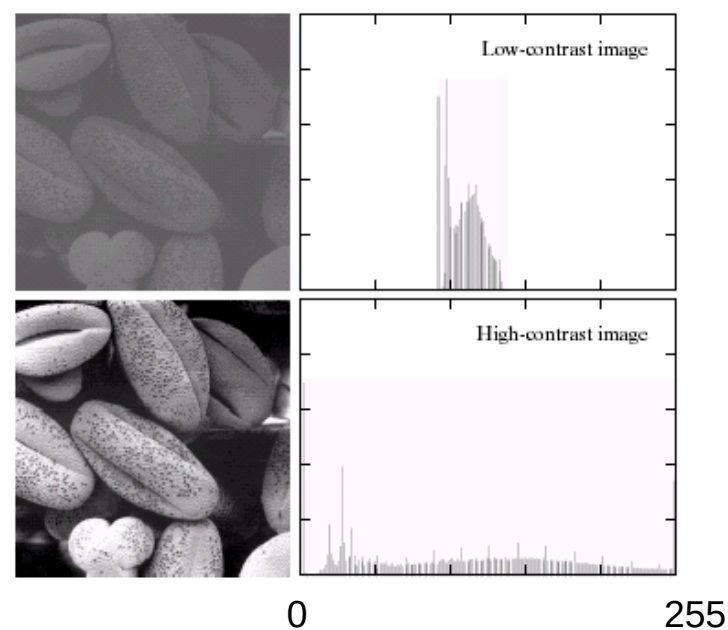
Point processing

- Image histogram 直方图

图像亮度



图像对比度



分度分布范围越广，
对比度越高

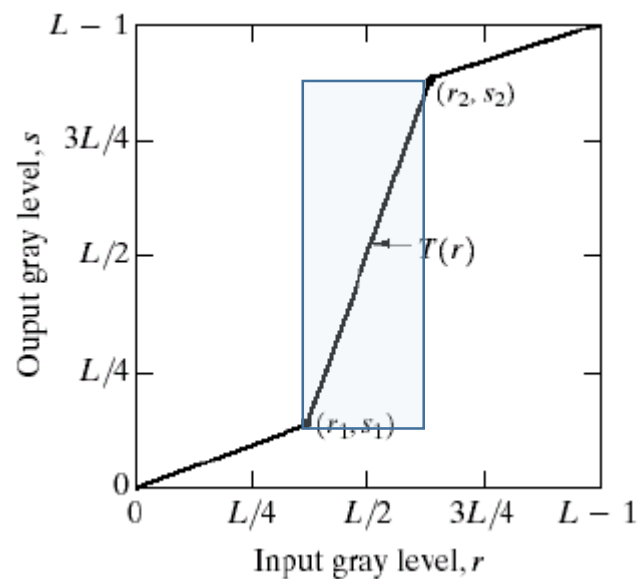
灰度值 i 的概率:

$$p_i = \frac{n_i}{n}$$

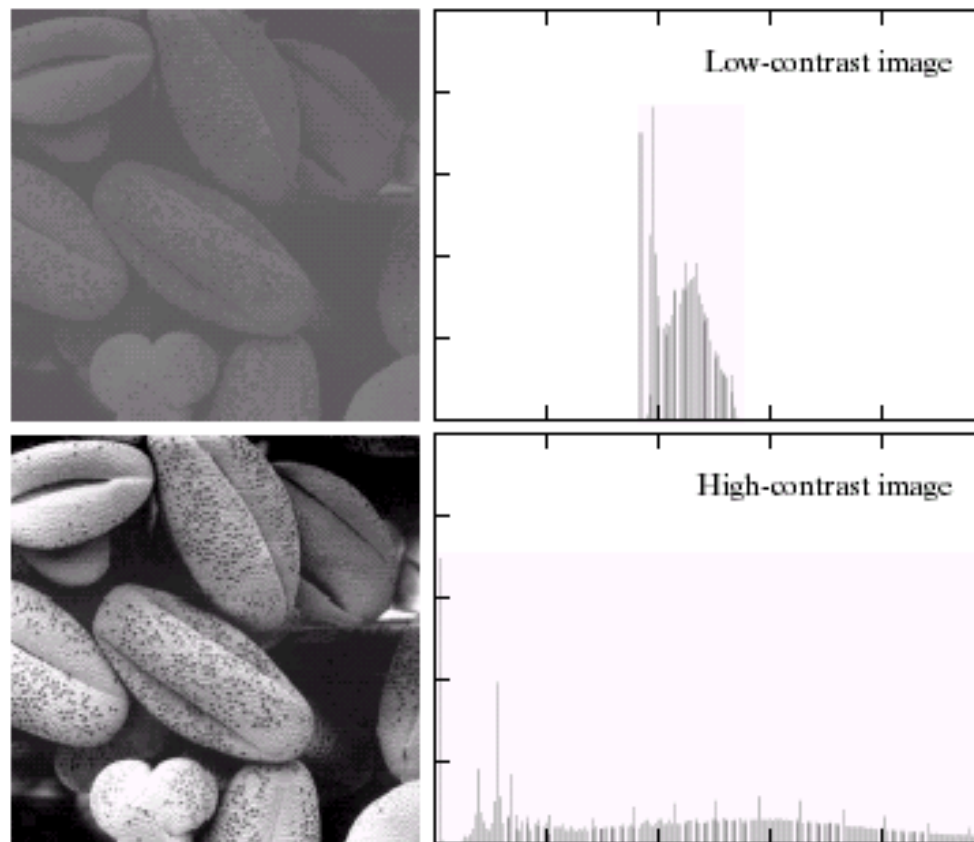
---灰度值为 i 的像素个数
---图像中所有像素的个数

Point processing

- Contrast stretching 对比度拉伸



Transformation function



After contrast stretching

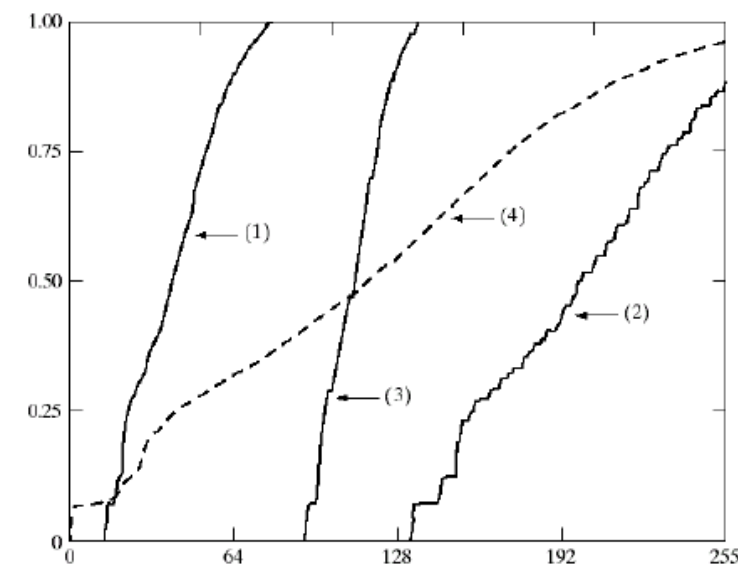
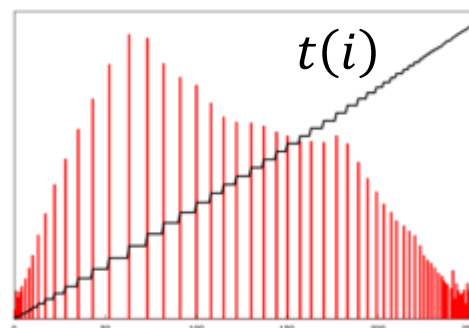
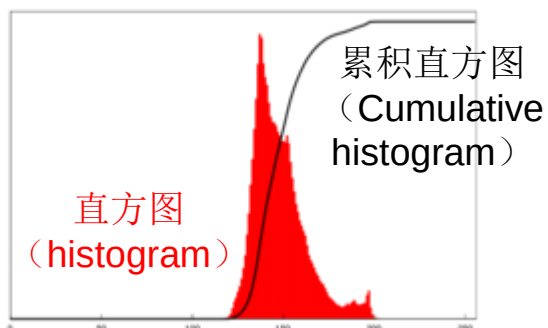
Point processing

- Histogram equalization 直方图均衡化
 - Make cumulative histogram (累积直方图) go from left bottom to top right
 - Improve the contrast

image



累积直方图是对直方图（概率分布）进行积分的结果

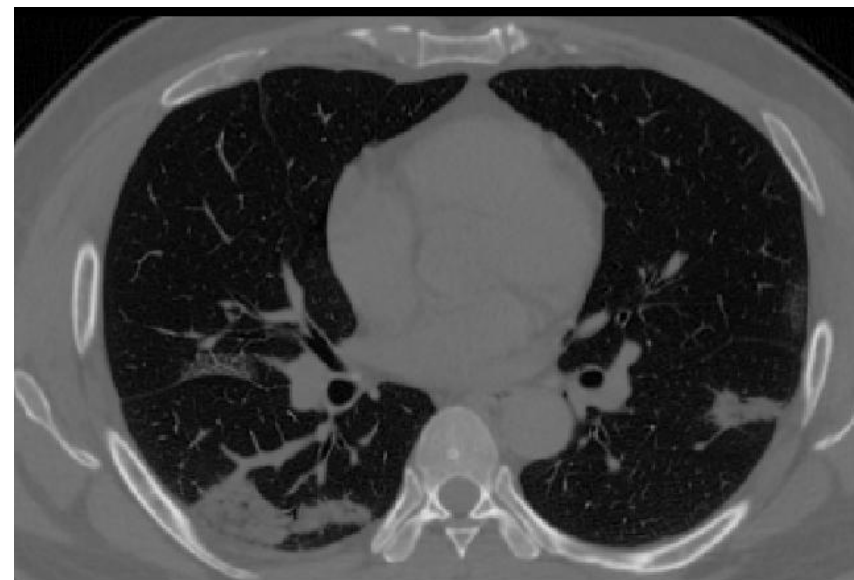
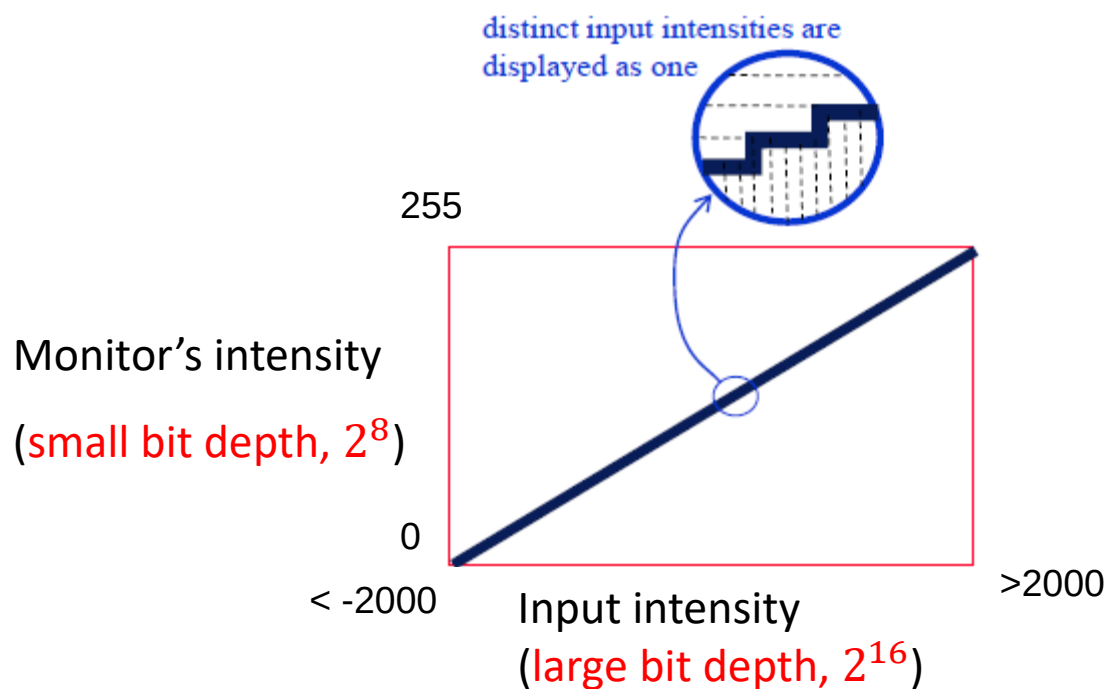


$$t(i) = \sum_{j=0}^i p(j) = \sum_{j=0}^i \frac{n_j}{n}$$

$$y_i = f(i) = (L - 1)t(i)$$

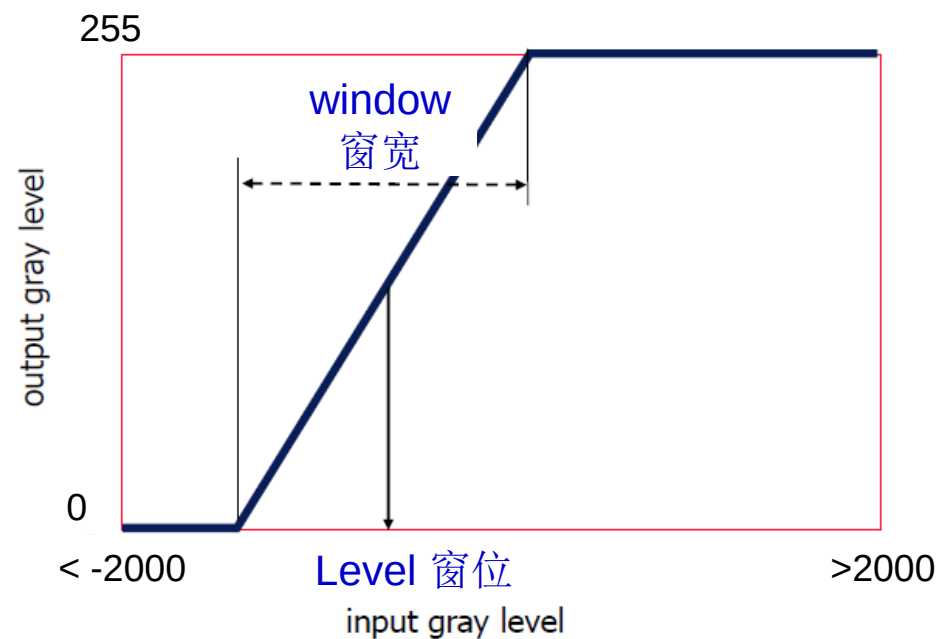
Point processing

- Window-level adjustment 窗口窗位调整
 - 医学图像 (CT/MRI) 具有较宽范围的灰度值, 如 $(-2000, 2000)$ 之间
 - 为了在屏幕上显示, 需将其映射到 $0-255$ (或 $0-1$) 之间



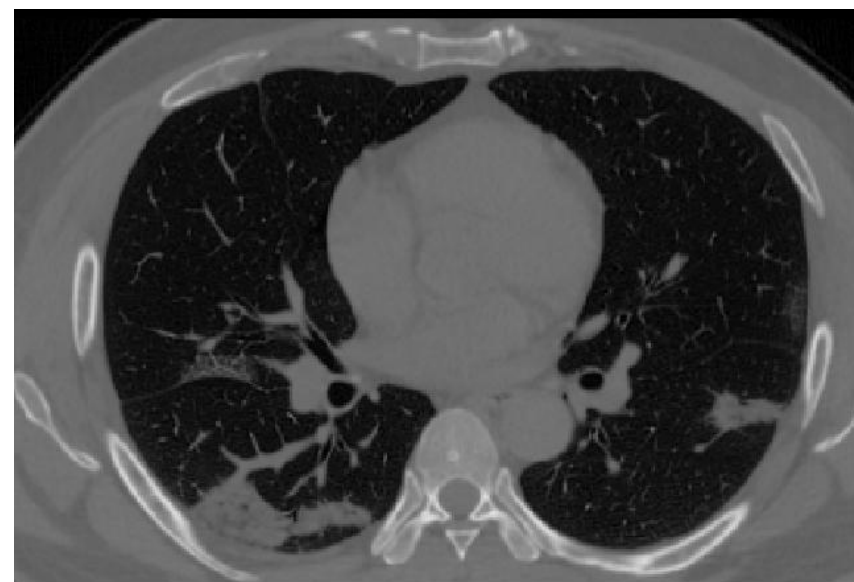
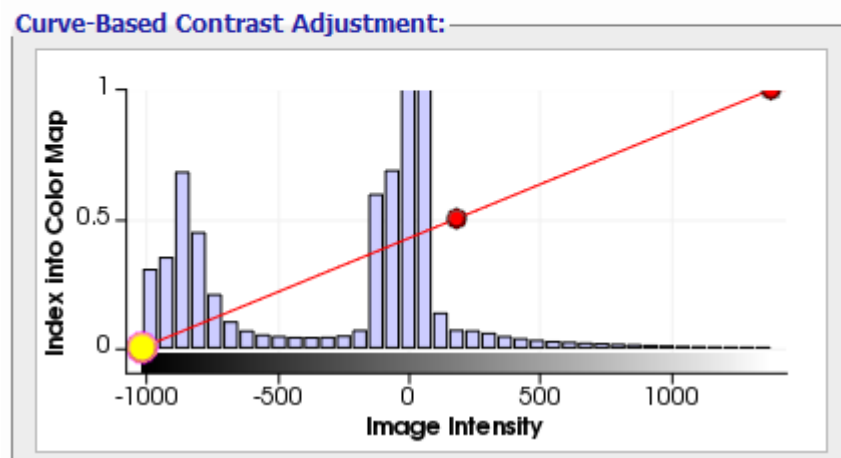
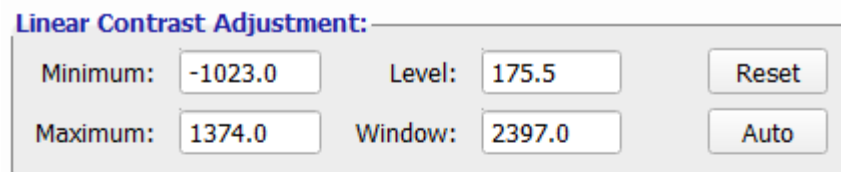
Point processing

- Window-level adjustment 窗口窗位调整
 - 医学图像 (CT/MRI) 具有较宽范围的灰度值, 如 $(-2000, 2000)$ 之间
 - 为了在屏幕上显示, 需将其映射到 $0-255$ (或 $0-1$) 之间



Point processing

- Window-level adjustment 窗口窗位调整
 - 医学图像 (CT/MRI) 具有较宽范围的灰度值, 如 $(-2000, 2000)$ 之间
 - 为了在屏幕上显示, 需将其映射到 $0-255$ (或 $0-1$) 之间



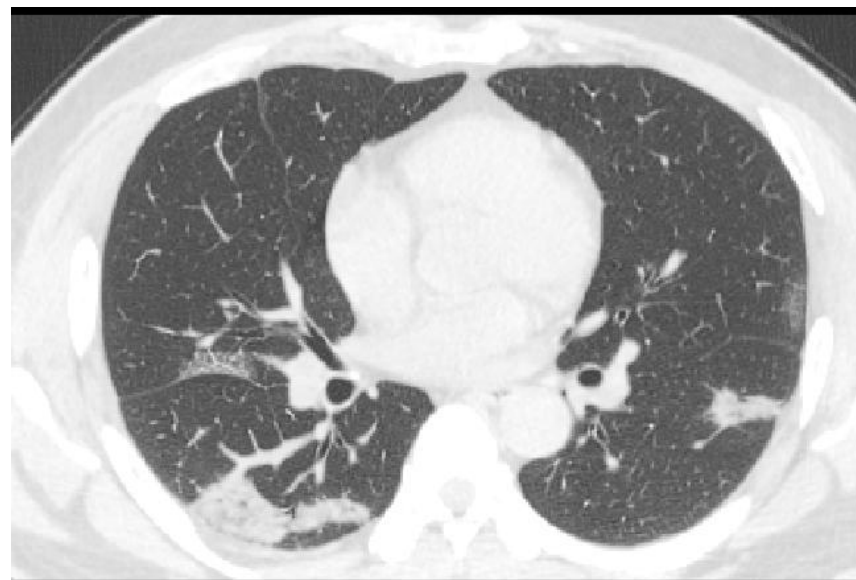
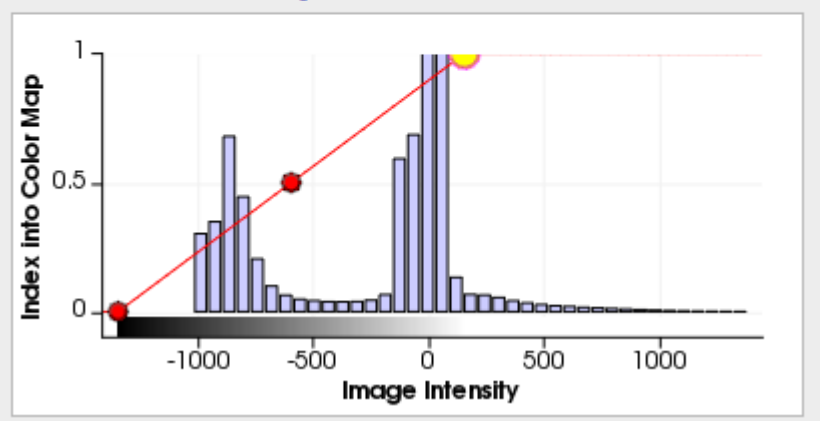
Point processing

- Window-level adjustment 窗口窗位调整
 - 医学图像 (CT/MRI) 具有较宽范围的灰度值, 如 $(-2000, 2000)$ 之间
 - 为了在屏幕上显示, 需将其映射到 $0-255$ (或 $0-1$) 之间

Linear Contrast Adjustment:

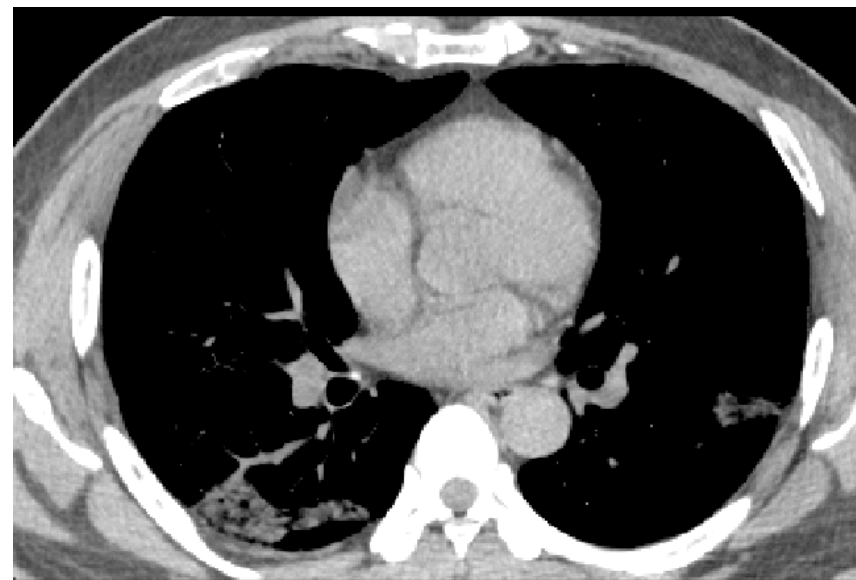
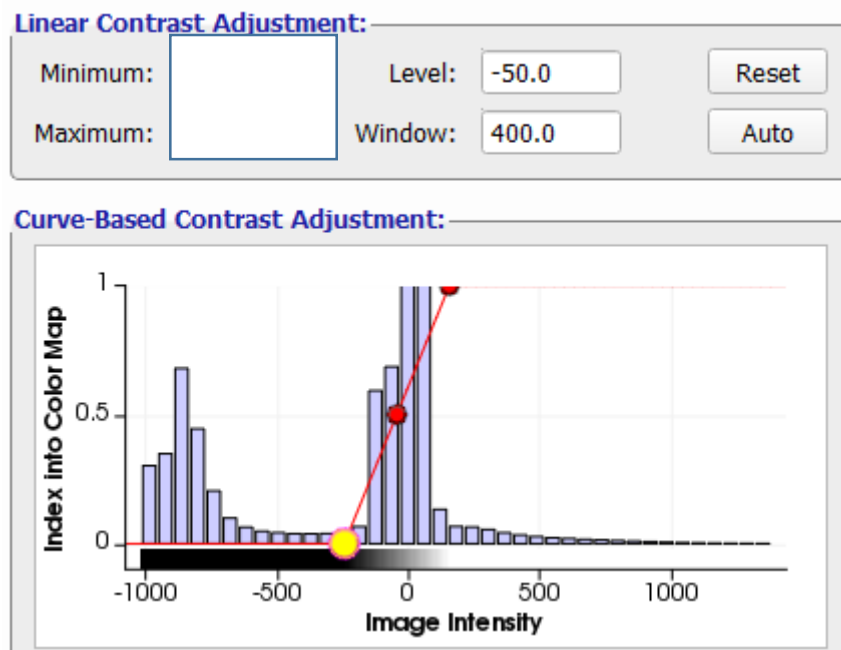
Minimum:	-1350.0	Level:	-600.0	Reset
Maximum:	150.0	Window:	1500.0	Auto

Curve-Based Contrast Adjustment:



Point processing

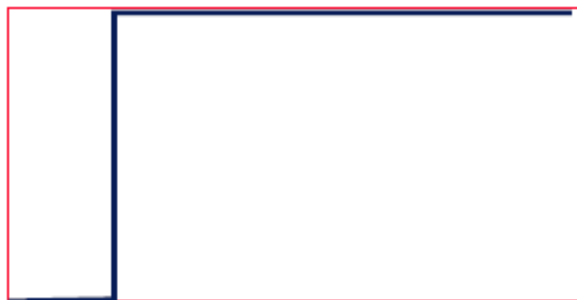
- Window-level adjustment 窗口窗位调整
 - 医学图像 (CT/MRI) 具有较宽范围的灰度值, 如 $(-2000, 2000)$ 之间
 - 为了在屏幕上显示, 需将其映射到 $0-255$ (或 $0-1$) 之间



Point processing

- Window-level adjustment 窗口窗位调整
 - 医学图像 (CT/MRI) 具有较宽范围的灰度值, 如 $(-2000, 2000)$ 之间
 - 为了在屏幕上显示, 需将其映射到 $0-255$ (或 $0-1$) 之间

Window = 0
Level = -200



如果窗宽为0, 得到的将是二值图像

Image processing basics



- Point processing
 - Gamma correction
 - Histogram equalization
 - Window-center correction
- Filtering 滤波
 - Mean, gradient
 - Convolution, cross-correlation
 - Gaussian and median filters
- Frequency analysis

Neighborhood processing (filtering)

- Mean filter 均值滤波

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$

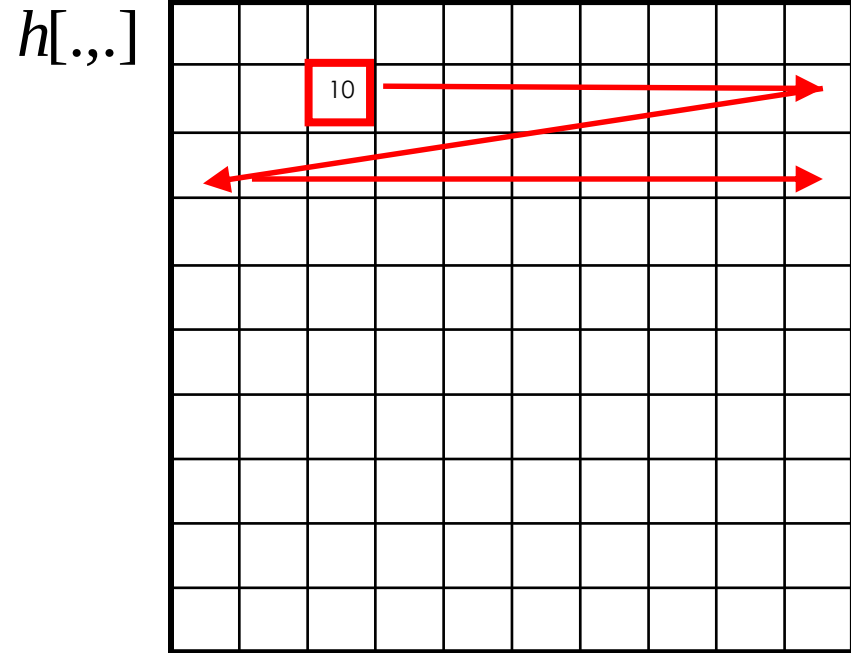
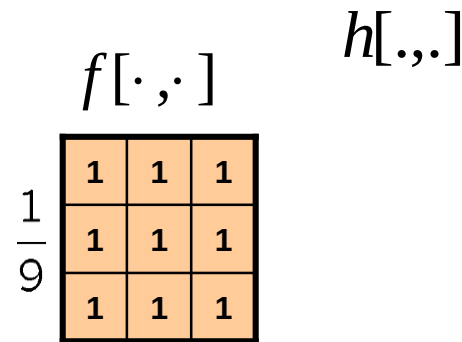
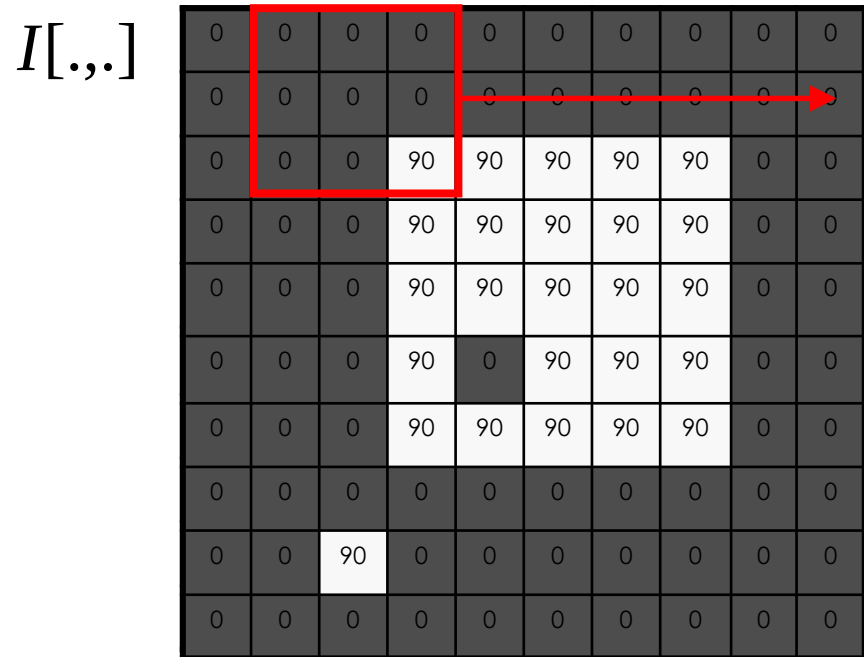
0									

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$$\begin{aligned} m &= 1, n = 1 \\ k, l &= [-1, 0, 1] \end{aligned}$$

Neighborhood processing (filtering)

- Mean filter 均值滤波

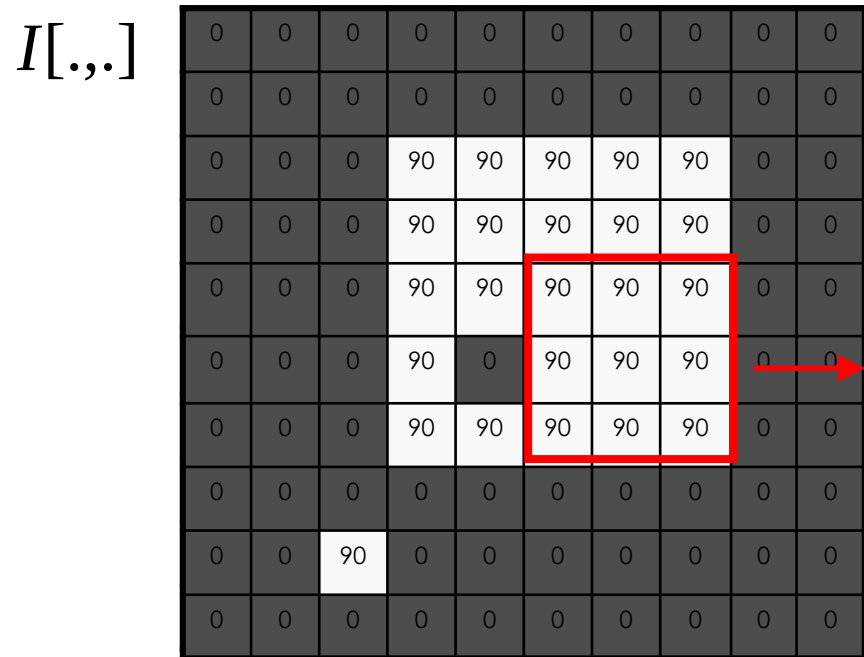


$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

$$\begin{aligned} m &= 2, n = 1 \\ k, l &= [-1, 0, 1] \end{aligned}$$

Neighborhood processing (filtering)

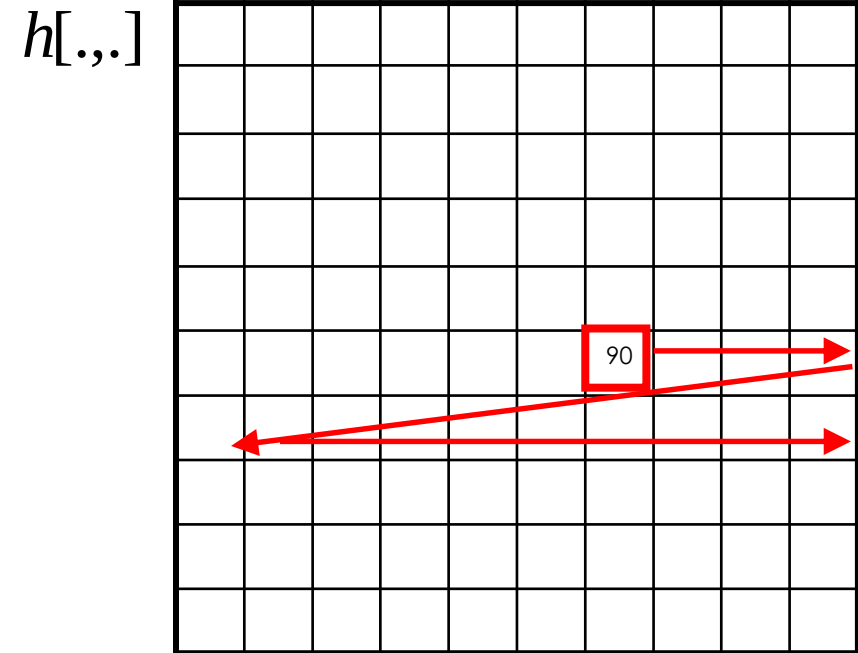
- Mean filter 均值滤波



$f[.,.]$

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$



Neighborhood processing (filtering)

- Mean filter 均值滤波

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$h[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

Neighborhood processing (filtering)

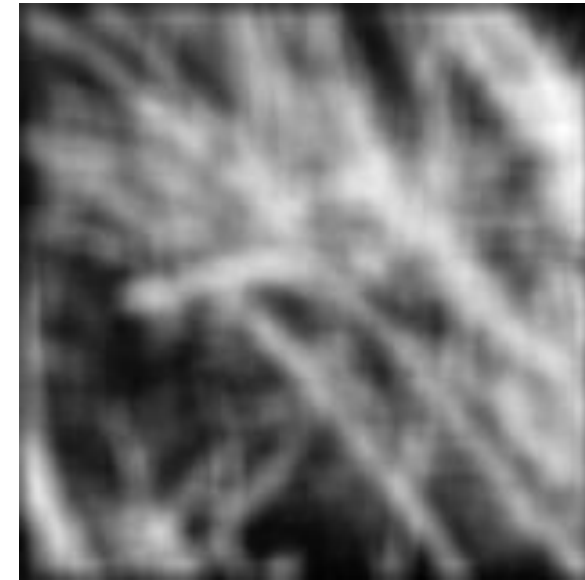
- Mean filter 均值滤波
- What does it do?
 - Replaces each pixel with an average of its neighborhood
 - Achieve smoothing effect (remove sharp features)

使图像变得模糊



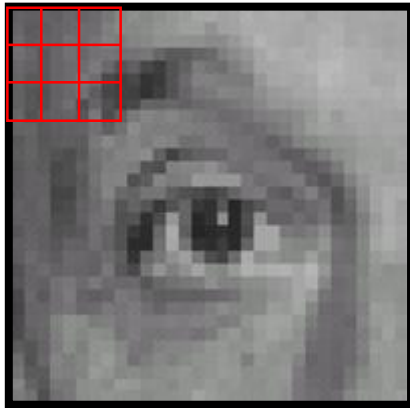
$$\frac{1}{9} f[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1



Neighborhood processing (filtering)

- Linear filter 线性滤波器



Original

0	0	0
0	1	0
0	0	0

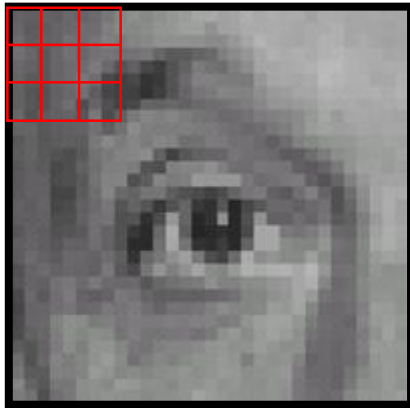


Filtered (no change)

$$h[m,n] = \sum_{k,l} f[k,l] I[m+k,n+l]$$

Neighborhood processing (filtering)

- Linear filter 线性滤波器



Original

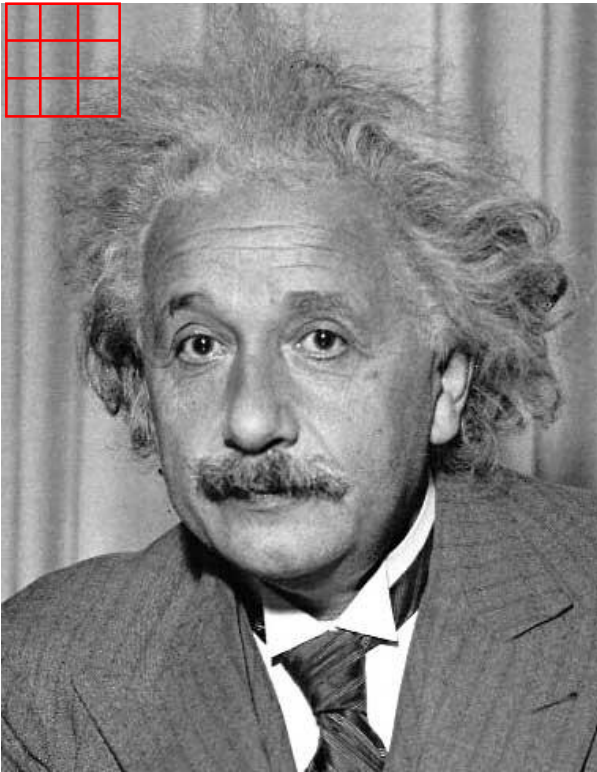
0	0	0
0	0	1
0	0	0



Shifted left By 1 pixel

Neighborhood processing (filtering)

- Linear filter 线性滤波器



1	0	-1
2	0	-2
1	0	-1

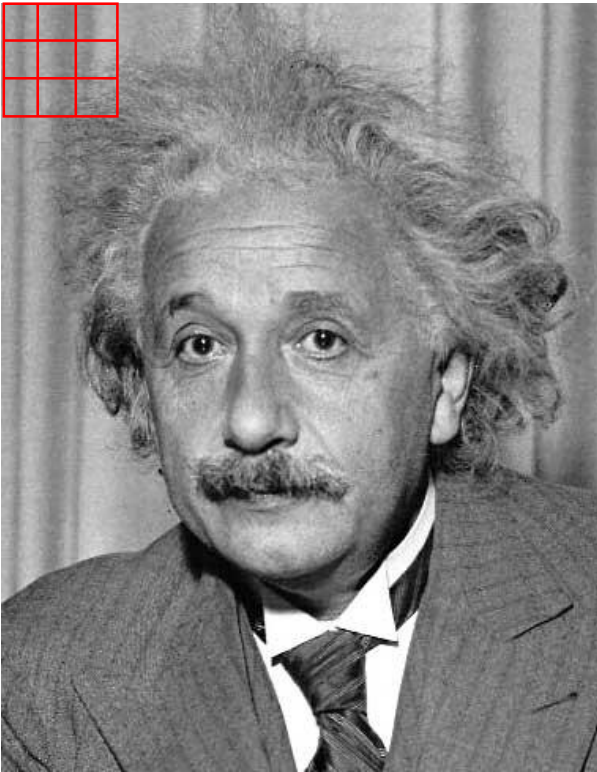
Sobel



Vertical Edge
(absolute value)

Neighborhood processing (filtering)

- Linear filter 线性滤波器



1	2	1
0	0	0
-1	-2	-1

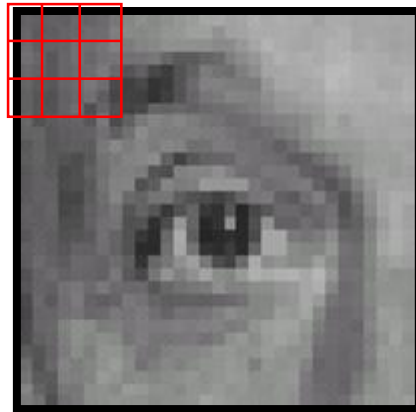
Sobel



Horizontal Edge
(absolute value)

Neighborhood processing (filtering)

- Linear filter 线性滤波器



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

(Note that filter sums to 1)



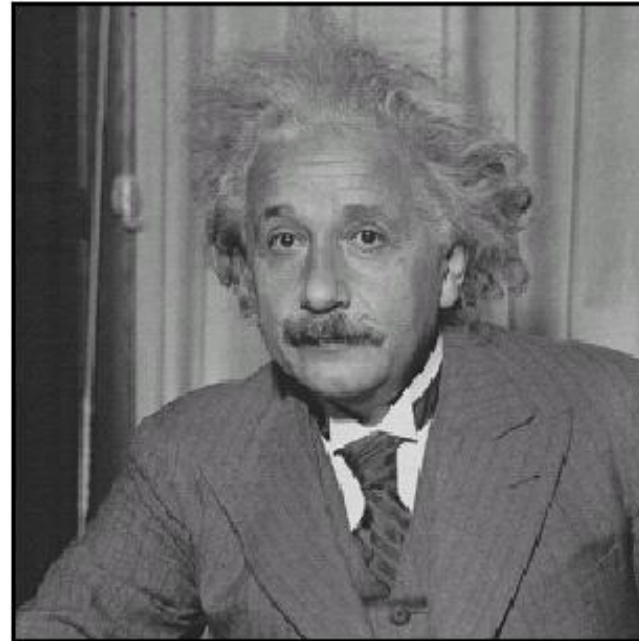
Sharpening filter 锐化

- Accentuates differences with local average

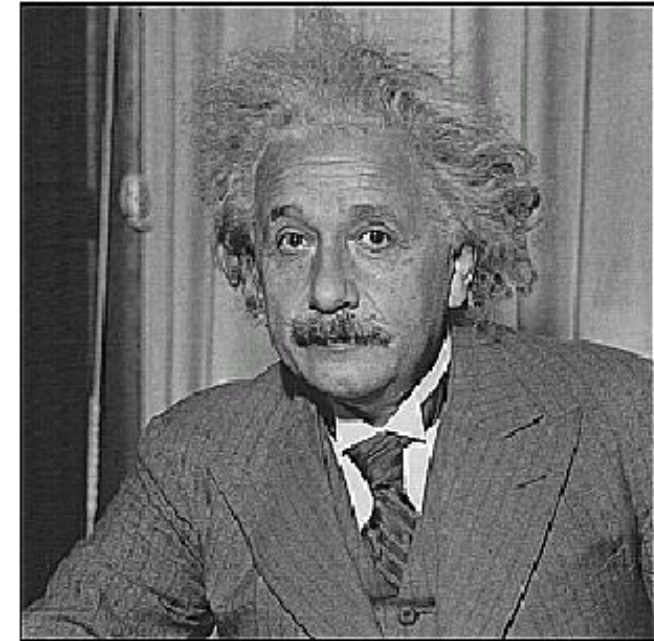
Neighborhood processing (filtering)

- Sharpening filter 锐化

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



before



after

Cross-correlation (自相关) and Convolution (卷积)



- A 2D image $I[m, n]$ can be filtered by a **2D kernel** $f[k, l]$ to produce an output image $h[m, n]$

自相关

$$h[m, n] = \sum_{k=-u}^u \sum_{l=-v}^v f[k, l] \cdot I[m + k, n + l]$$

$$\frac{1}{9} f[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

- This is also called **cross-correlation** operation

$$h = I \circ f$$

输出图像 输入图像 滤波核

f is called kernel, or mask, or filter which represents a given window function

Cross-correlation (自相关) and Convolution (卷积)



- Closely related **convolution** operation is defined slightly different
 - Filter is **flipped** both horizontally and vertically before being applied to the image

卷积

$$h[m, n] = \sum_{k=-u}^u \sum_{l=-l}^l \overset{\text{滤波核进行翻转}}{f[-k, -l]} \cdot I[m + k, n + l]$$

$f[\cdot, \cdot]$

	1	1	1
1	1	1	1
1	1	1	1

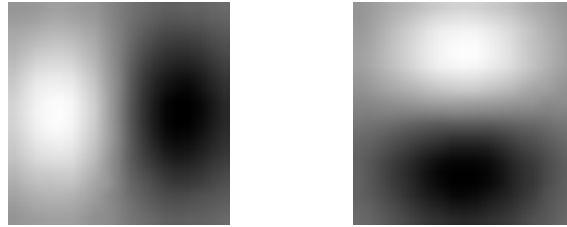
- It is written as $h = I * f$ 卷积
 - If $f[k, l] = f[-k, -l]$, then convolution is the same as cross-correlation
 - In practice, convolution and cross-correlation are often used indiscriminately
 - For example, CNNs implementations often use cross-correlations

NCC for Template matching 模板匹配



- Filters as **templates**:

Note that filters look like the effects they are intended to find --- “matched filters”



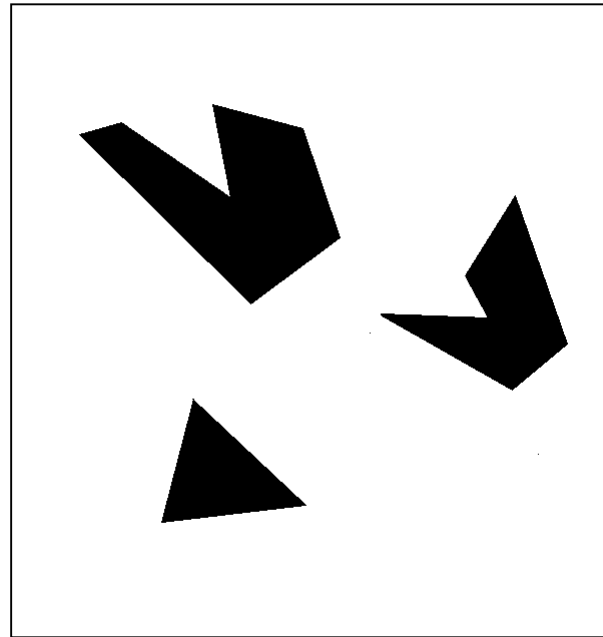
归一化自相关

- **Normalized cross-correlation (NCC)** to find a given pattern (template) in the image.
- Normalization needed to control for relative brightness.

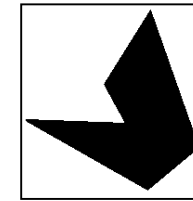
NCC for Template matching 模板匹配



- Example
- Normalized cross-correlation (NCC) to find a given pattern (template) in the image



Scene

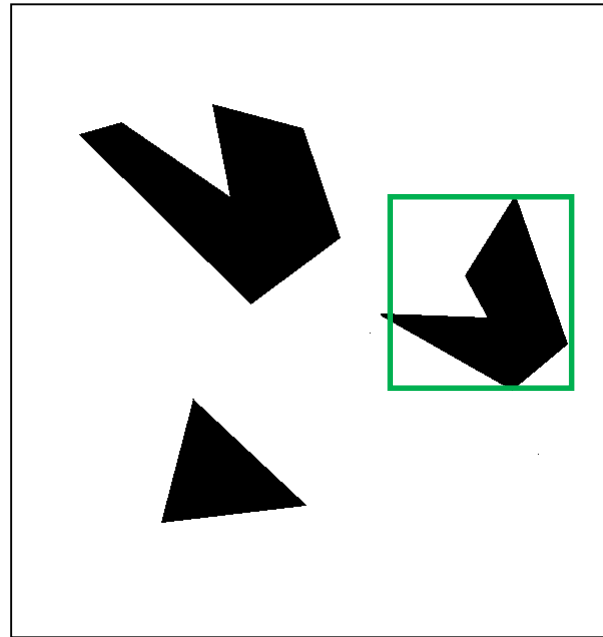


Template (mask)

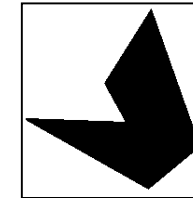
NCC for Template matching 模板匹配



- Example
- Normalized cross-correlation (NCC) to find a given pattern (template) in the image



Detected template

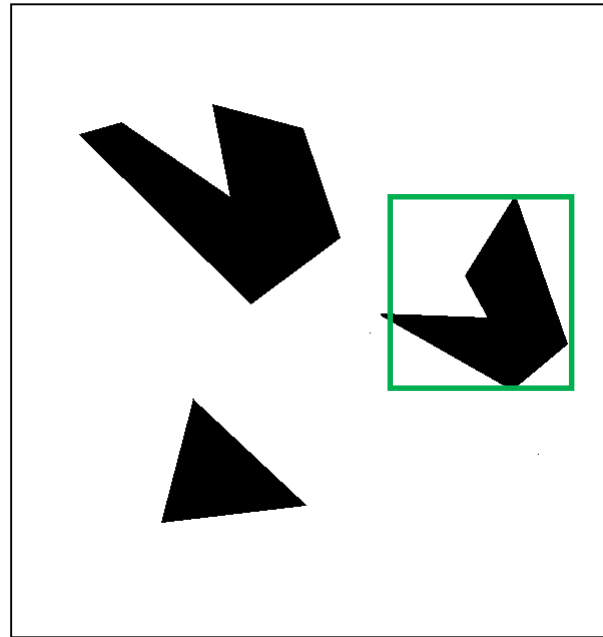


Template (mask)

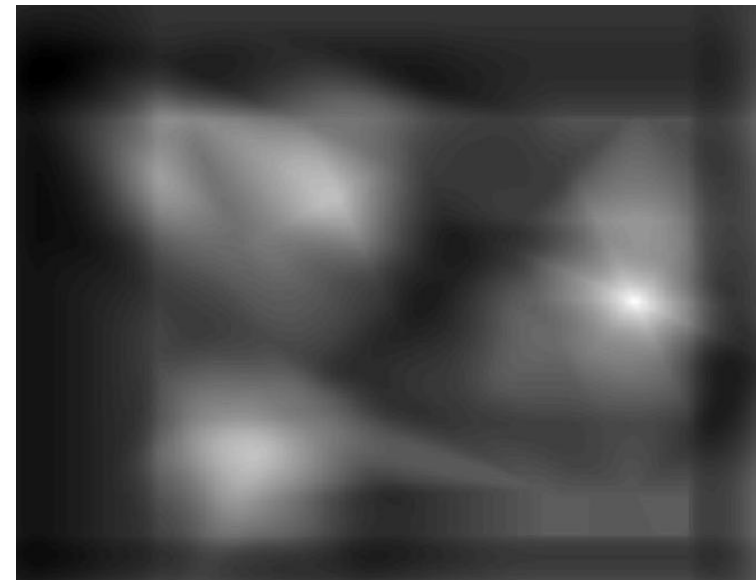
NCC for Template matching 模板匹配

- Example

Normalized cross-correlation (NCC) to find a given pattern (template) in the image



Detected template



Correlation map

$$h[m, n] = \sum_{u=-k}^k \sum_{v=-k}^k f[u, v] \cdot I[m + u, n + v]$$

NCC for Template matching

- Example: Where is Wally?



Scene



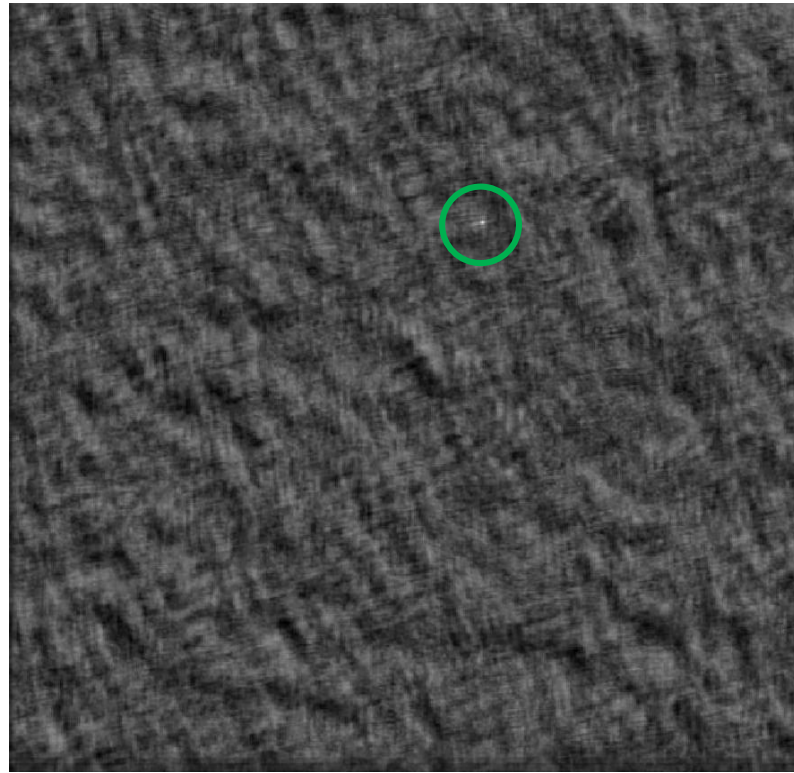
Template

NCC for Template matching

- Example: Where is Wally?



Scene



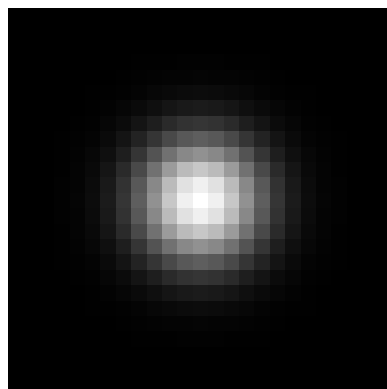
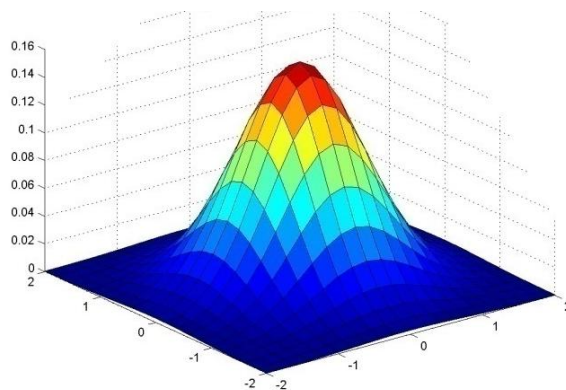
Correlation map



Template

Gaussian filtering 高斯滤波

- Important filter: Gaussian
 - Weight contributions of neighboring pixels by nearness



$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

X				
0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003
Y				

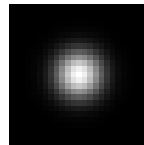
5 x 5, $\sigma = 1$

Gaussian filtering 高斯滤波

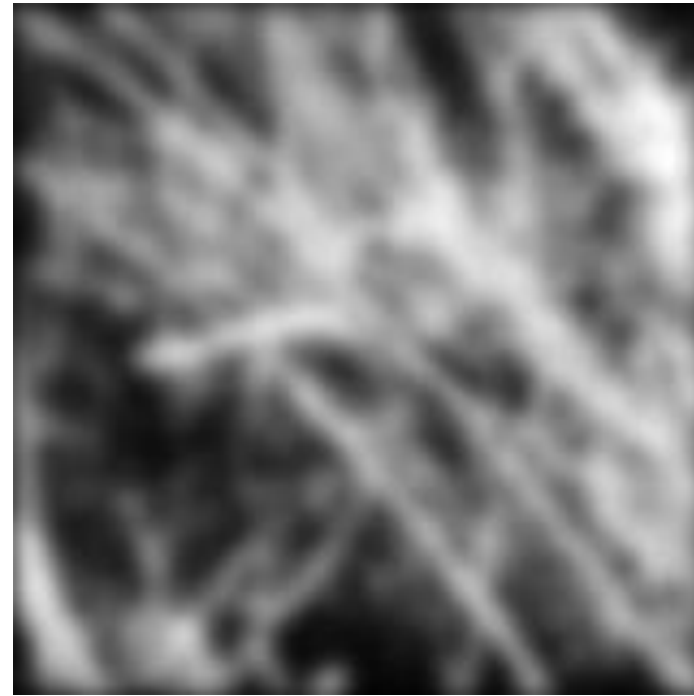
- Important filter: Gaussian
 - Weight contributions of neighboring pixels by nearness



input



kernel



output

Gaussian filtering 高斯滤波



Properties of Gaussian filters

- Remove “high-frequency” components from the image (**low-pass filter**)
低通滤波器
 - Images become more smooth
- Gaussian convolved with Gaussian is another Gaussian
 - So can smooth with small-width kernel, repeat, and get same result as larger-width kernel would have
 - Convolution twice with Gaussian kernel of width σ is same as convolving once with kernel of width $\sigma\sqrt{2}$

两个高斯函数的卷积仍然是高斯函数
- **Separable** kernel
 - Factors into product of two 1D Gaussians

高斯函数在不同维度上具有可分离特性

Gaussian filtering 高斯滤波



Separability of the Gaussian filter

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the project of two functions, one a function of x and the other a function of y

Gaussian filtering 高斯滤波



Separability example

2D convolution
(center location
only)

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 3 & 3 \\ 3 & 5 & 5 \\ 4 & 4 & 6 \end{bmatrix} =$$

The filter factors into a
product of 1D filters:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Perform convolution
along rows:

$$\begin{bmatrix} 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 3 & 3 \\ 3 & 5 & 5 \\ 4 & 4 & 6 \end{bmatrix} = \begin{bmatrix} & 11 & \\ & 18 & \\ & 18 & \end{bmatrix}$$

Followed by convolution
along the remaining column:

Gaussian filtering 高斯滤波



Separability example

2D convolution
(center location
only)

1	2	1
2	4	2
1	2	1

*

2	3	3
3	5	5
4	4	6

$$= 2 + 6 + 3 = 11$$

$$= 6 + 20 + 10 = 36$$

$$= 4 + 8 + 6 = 18$$

65

=

	65	

The filter factors into a
product of 1D filters:

1	2	1
2	4	2
1	2	1

=

1
2
1

x

1	2	1
---	---	---

Perform convolution
along rows:

1	2	1
---	---	---

*

2	3	3
3	5	5
4	4	6

=

	11	
	18	
	18	

Followed by convolution
along the remaining column:

1
2
1

*

	11	
	18	
	18	

=

	65	

Gaussian filtering 高斯滤波



Separability

Why is separability useful in practice?

MxN image, PxQ filter

- 2D convolution: $\sim MN PQ$ multiply-adds
- Separable 2D: $\sim MN(P+Q)$ multiply-adds

Speed up = $PQ/(P+Q)$

9x9 filter = $\sim 4.5x$ faster

根据高斯核的可分离性，可将一个大的二维高斯核分解为x和y两个方向上的一维高斯核进行卷积操作；

可大大提高计算效率。

Neighborhood processing (filtering)



Convolution properties

Commutative: $a * b = b * a$ 交换律

- Conceptually no difference between filter and signal
- But particular filtering implementations might break this equality, e.g., image edges

Associative: $a * (b * c) = (a * b) * c$ 结合律

- Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
- This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$

Neighborhood processing (filtering)



高斯差分滤波器

Difference of Gaussian (DoG) filter

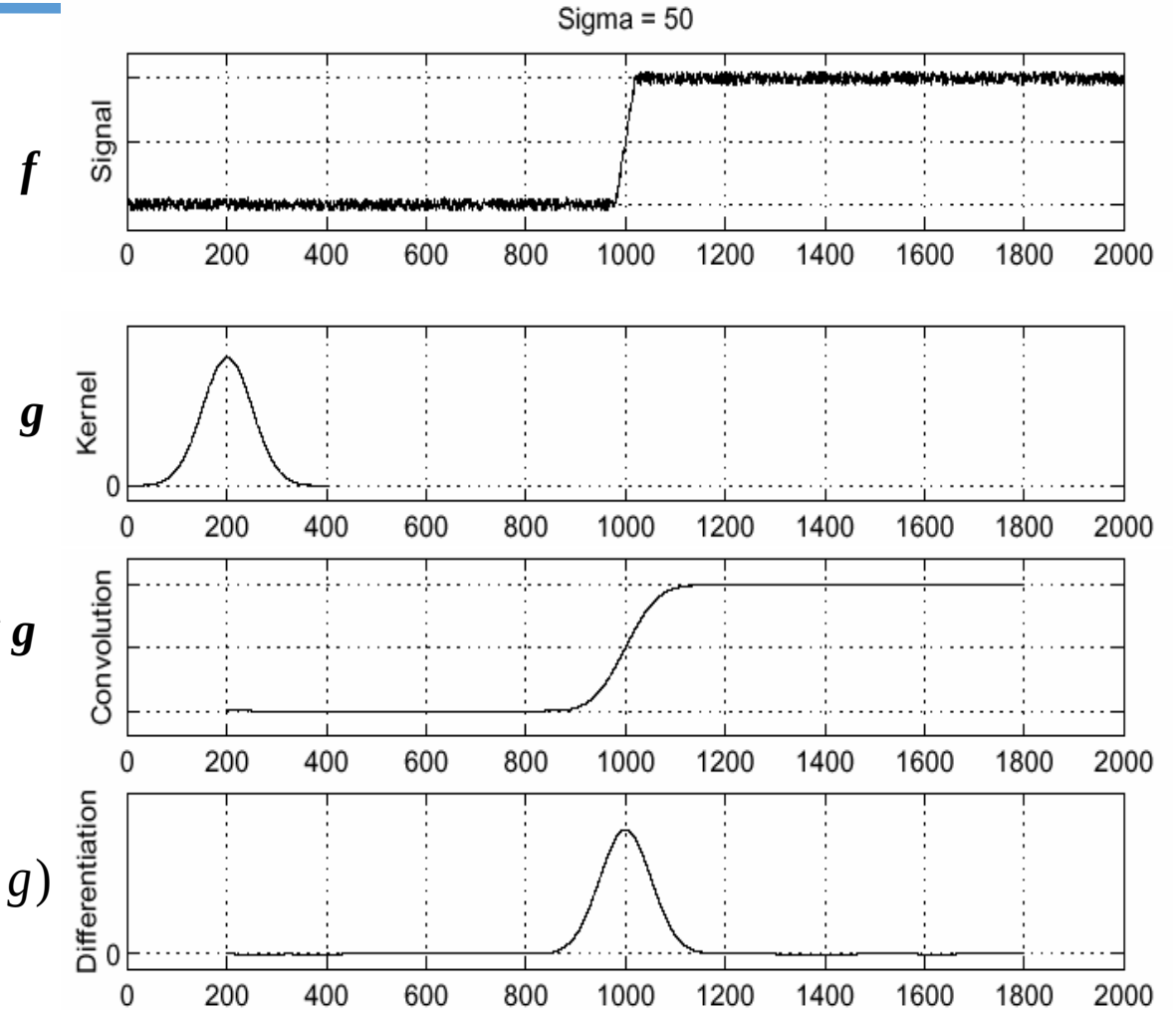
Consider the following signal, we first smooth it and then find the edge

g is a Gaussian filter for smoothing

d is difference operation (差分)

$$\frac{d}{dx}(f * g)$$

$$\frac{d}{dx}(f * g)$$



Neighborhood processing (filtering)



高斯差分滤波器

Difference of Gaussian (DoG) filter

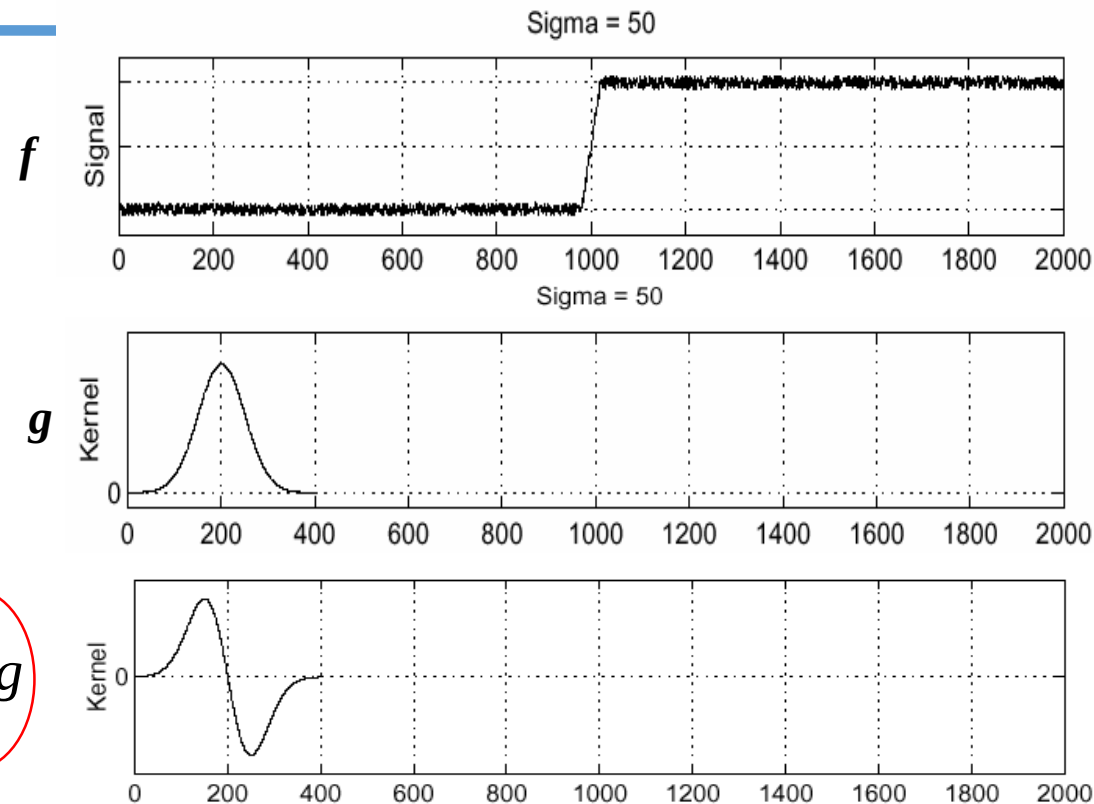
Consider the following signal, we first smooth it and then find the edge

g is a Gaussian filter for smoothing

d is difference operation (差分)

To improve efficiency, we can alternatively use **difference of Gaussian** for filtering

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$



Neighborhood processing (filtering)



高斯差分滤波器

Difference of Gaussian (DoG) filter

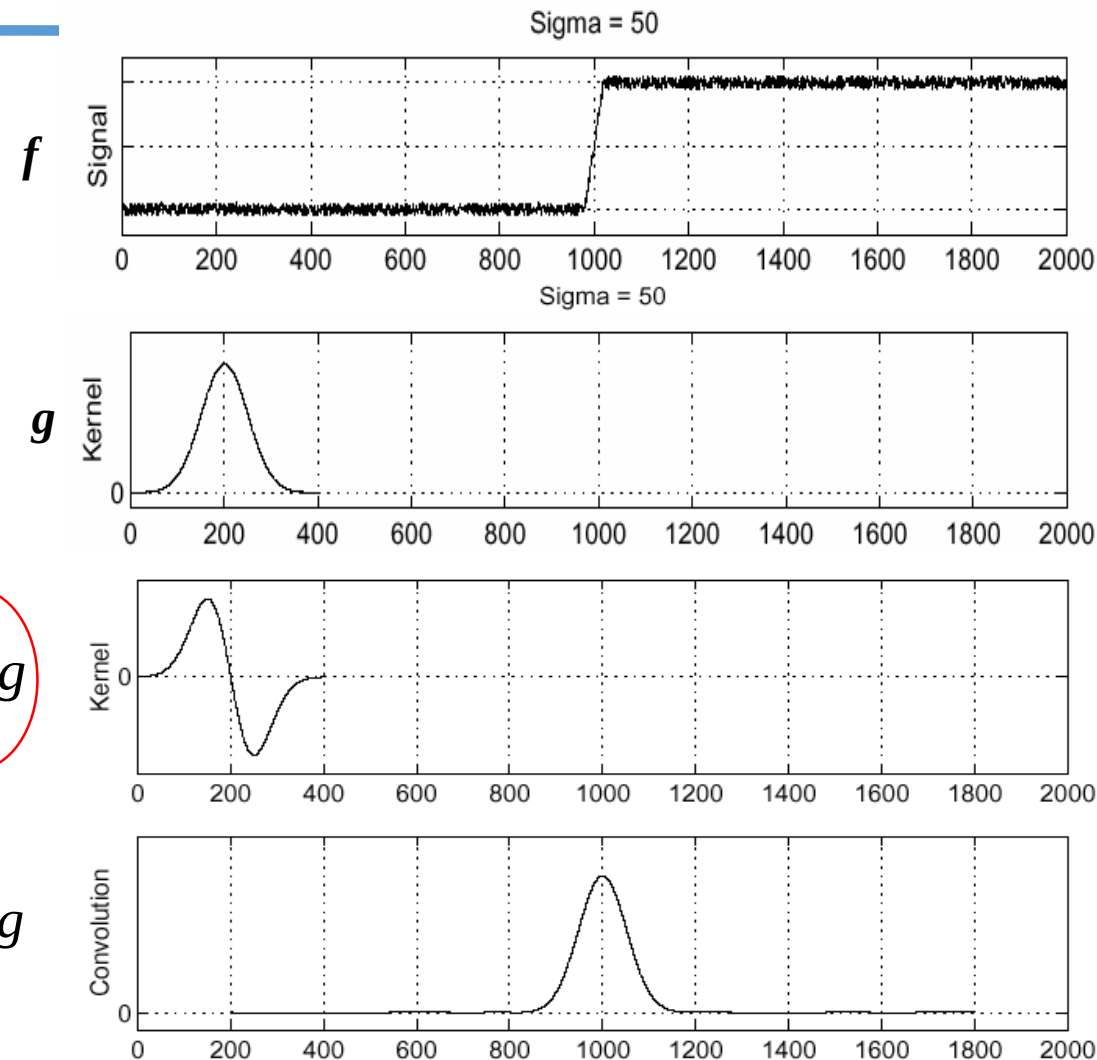
Consider the following signal, we first smooth it and then find the edge

g is a Gaussian filter for smoothing

d is difference operation (差分)

To improve efficiency, we can alternatively use **difference of Gaussian** for filtering $f * \frac{d}{dx} g$

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx} g$$

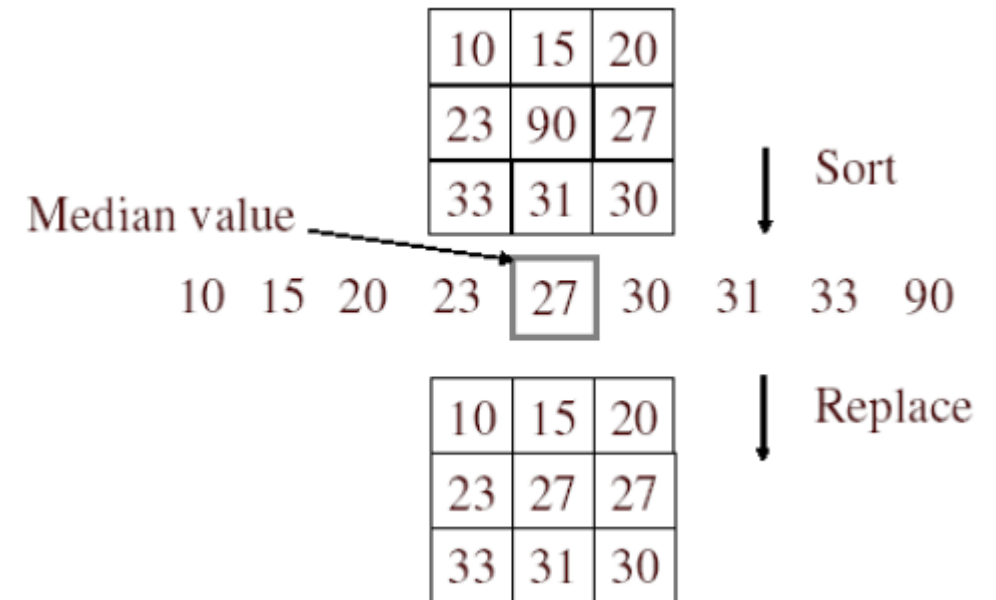


Neighborhood processing (filtering)



Median filters (中值滤波)

- Operates over a window by selecting the **median** intensity in the window.
- 'Rank' filter as based on ordering of gray levels
 - E.G., min, max, range filters
- Can better deal with salt and pepper noise than mean filter



- 将窗口中心位置的像素值替换为该窗口内的中位数
- 可更好处理椒盐噪声

Neighborhood processing (filtering)

Median filters (中值滤波)



Noise – Salt and Pepper Jack



Mean Jack – 3 x 3 filter



Median Jack – 3 x 3 filter

Neighborhood processing (filtering)



中值滤波可以保留边缘

Median filter is edge preserving

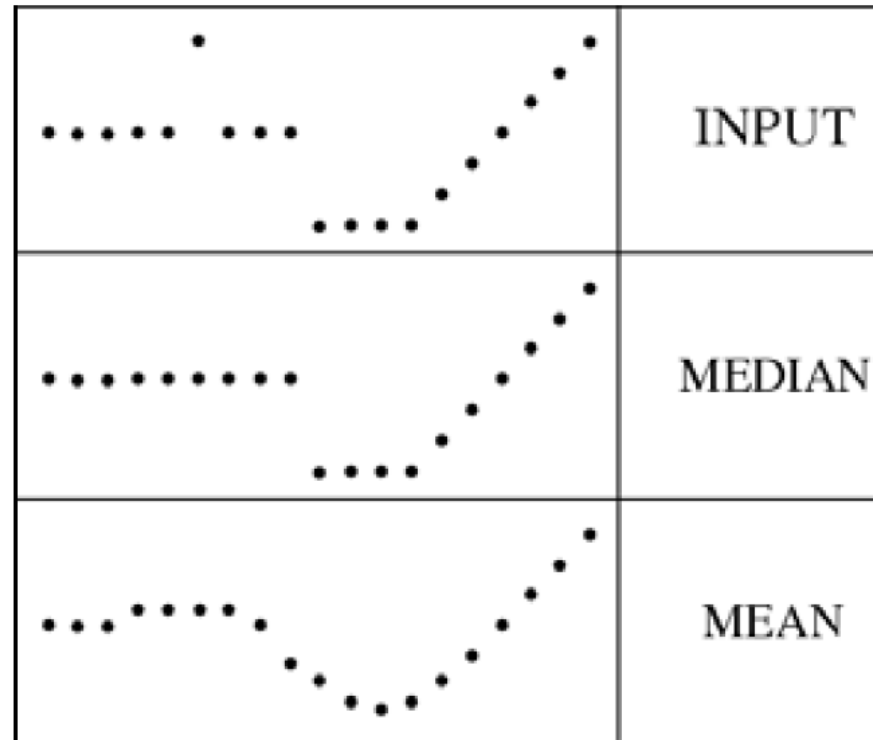


Image processing basics



- Point processing
 - Gamma correction
 - Histogram equalization
 - Window-center correction
- Filtering
 - Mean, gradient
 - Convolution, cross-correlation
 - Gaussian and median filters
- Frequency analysis 频率分析

Frequency Analysis 频率分析



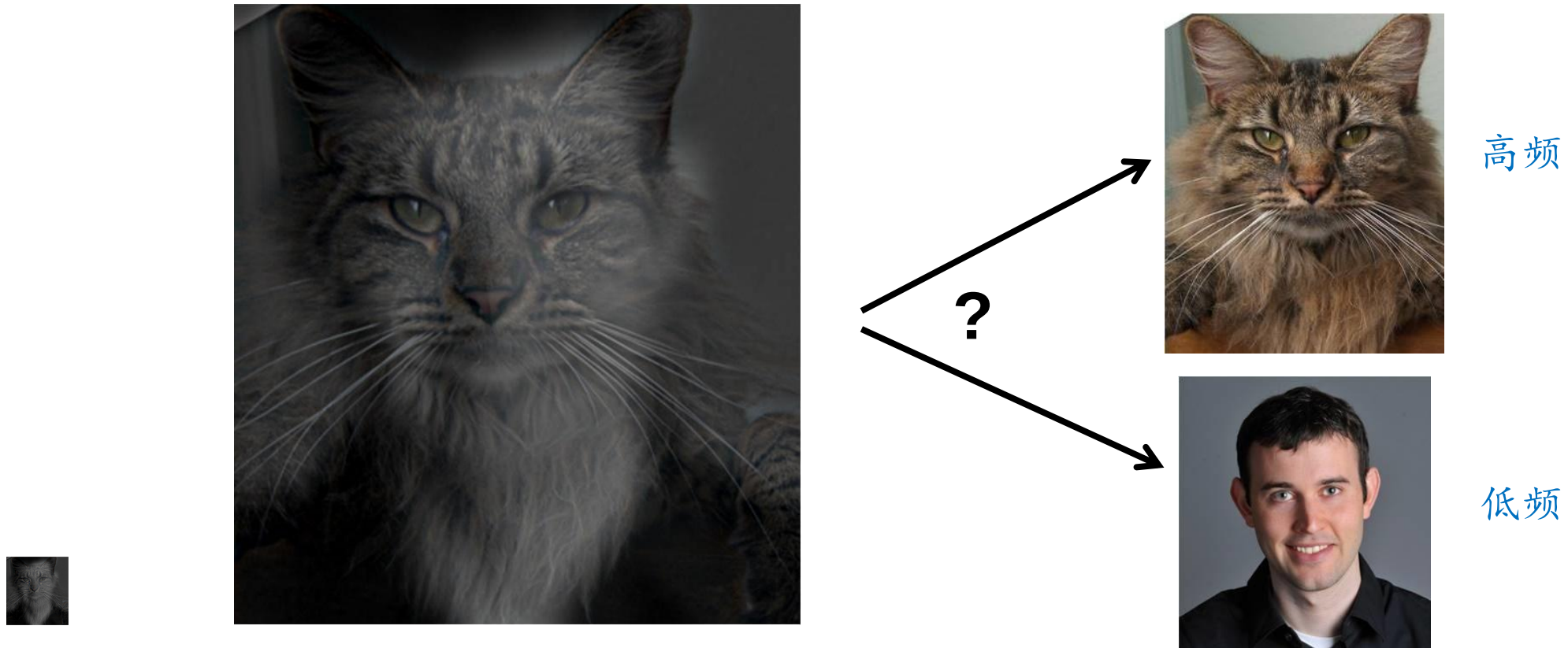
Distance-dependent interpretations of hybrid images



Frequency Analysis 频率分析



Distance-dependent interpretations of hybrid images



Frequency Analysis

Fourier series 傅里叶级数

Any univariate function can be rewritten as a weighted sum of **sines** and **cosines** of different frequencies.

Our building block:

$$\sum_{n=1}^{\infty} (a_n \cos(nt) + b_n \sin(nt))$$

Add enough of them to get any signal $g(t)$ you want!



Jean Baptiste Joseph Fourier (1768-1830)

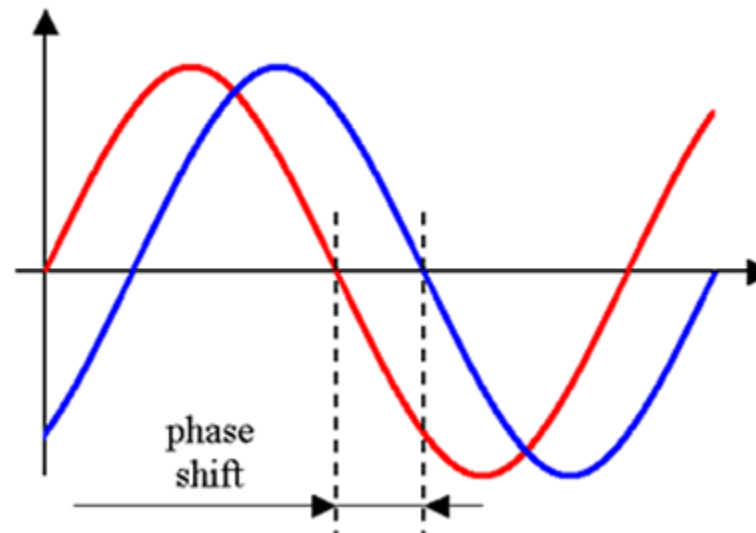
Frequency Analysis

Amplitude-phase form 幅值和相位的表达式

Add phase term to **shift cos values into sin values**

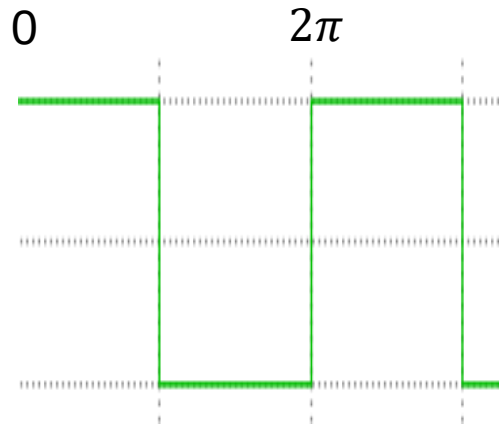
$$\sum_{n=1}^N (a_n \sin(nx + \phi_n))$$

Phase

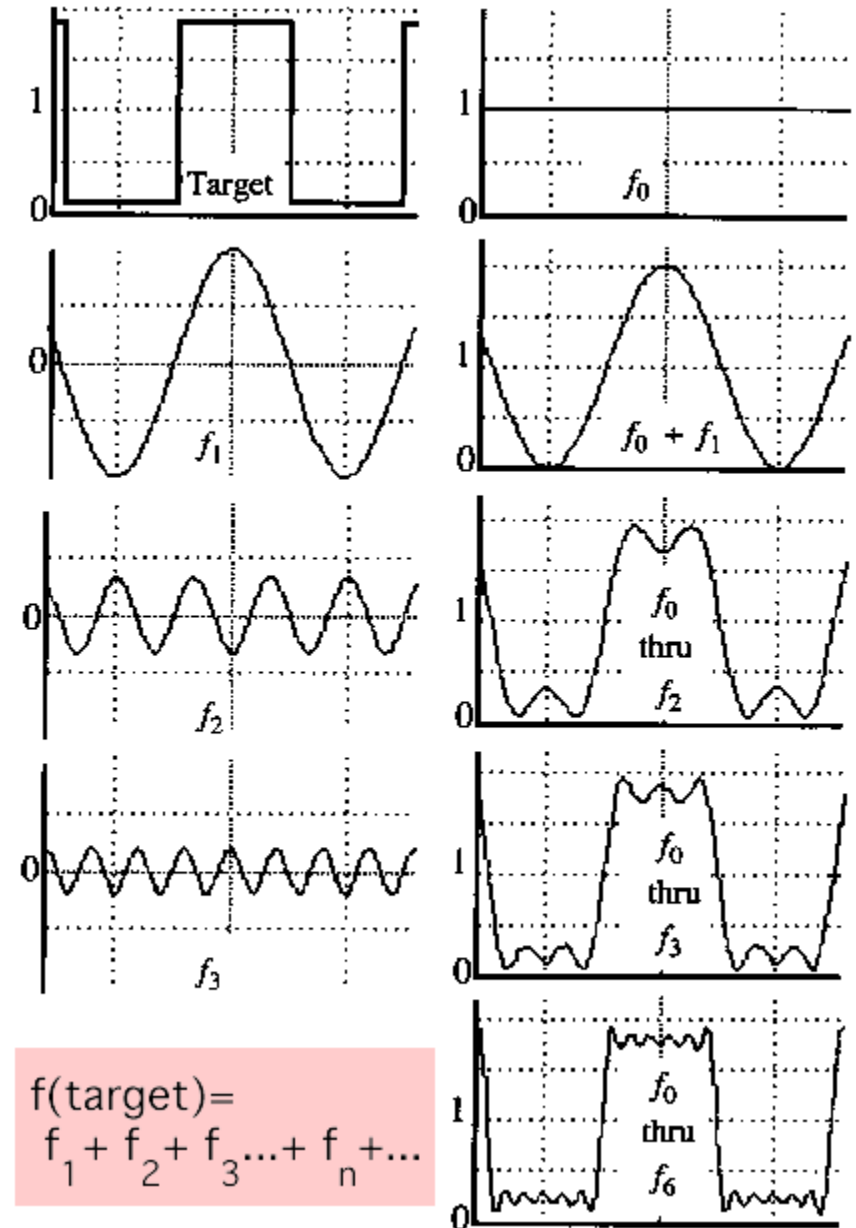
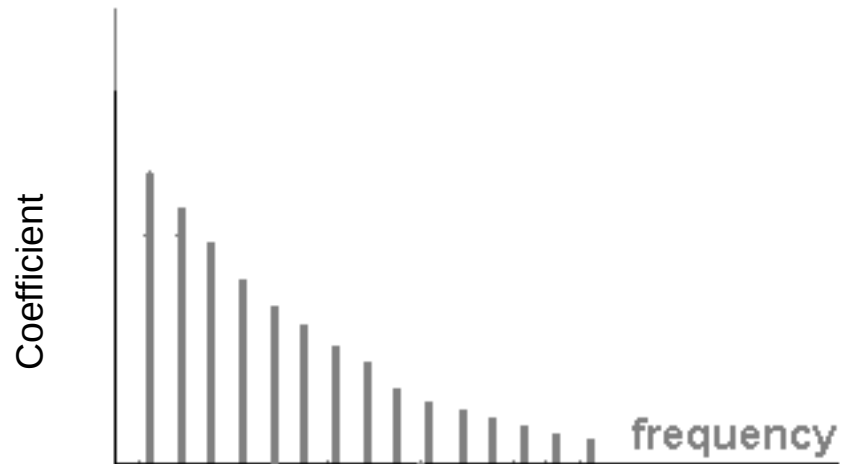


Frequency Analysis

Fourier series 傅里叶级数



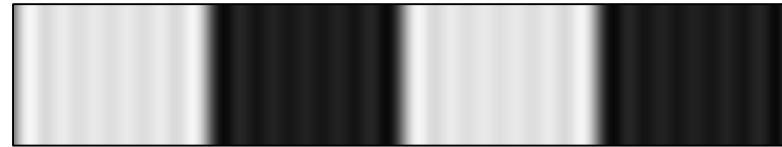
$$= \sum_{n=1}^{\infty} \frac{1}{n} \sin(nt)$$



Frequency Analysis

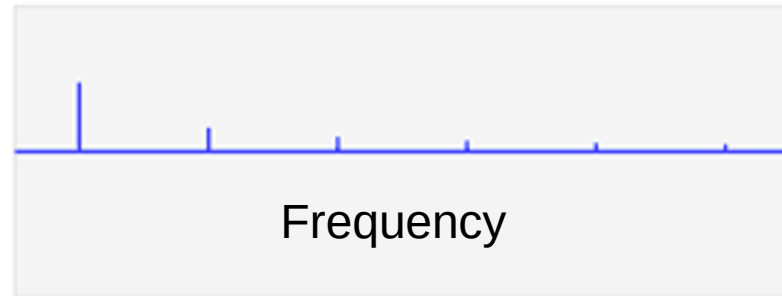
Example

空间域



Spatial
domain

Amplitude



Frequency
domain

频率域



Equivalent
amplitude image
representation

Frequency Analysis



Amplitude-phase form

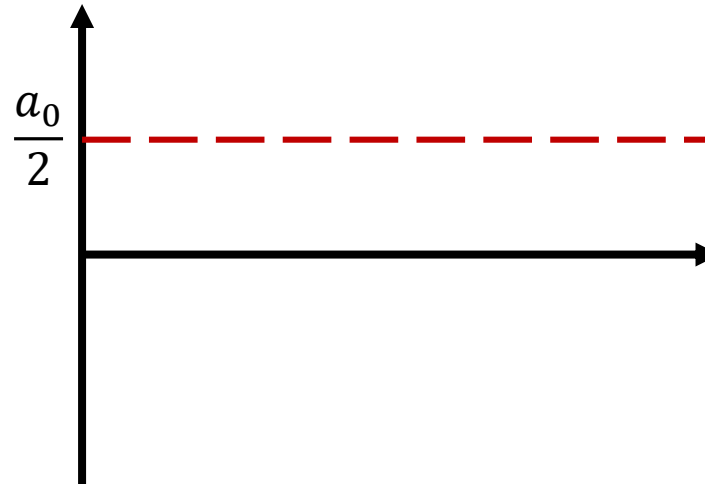
Add component of infinite frequency

= mean of signal over period

= value around which signal fluctuates

$$\frac{a_0}{2} + \sum_{n=1}^N (a_n \sin(nx + \phi_n))$$

Average of signal
over period



Frequency Analysis



How to read Fourier transform images

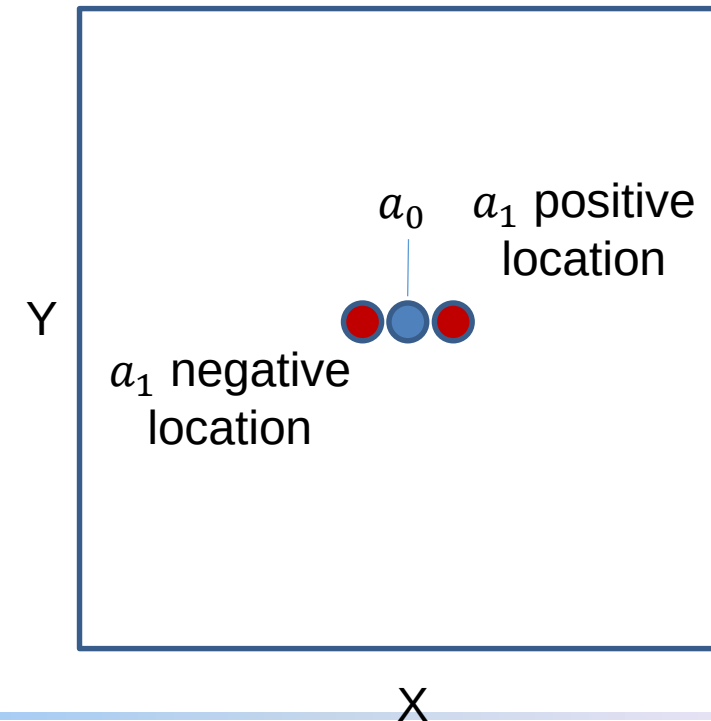
图像的傅里叶变换

Image is rotationally symmetric about center because of negative frequencies

$$\frac{a_0}{2} + \sum_{n=1}^N (a_n \sin(nx + \phi_n))$$

For real-valued signals, positive and negative frequencies are complex conjugates

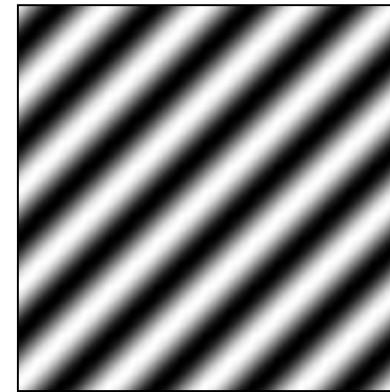
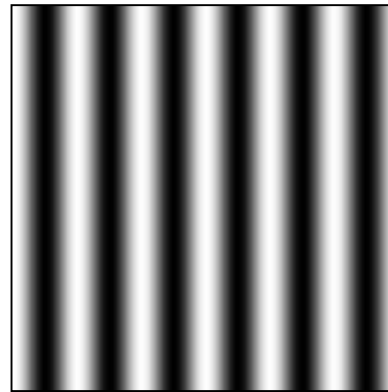
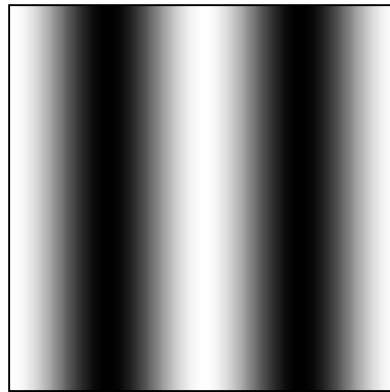
Fourier transform component image



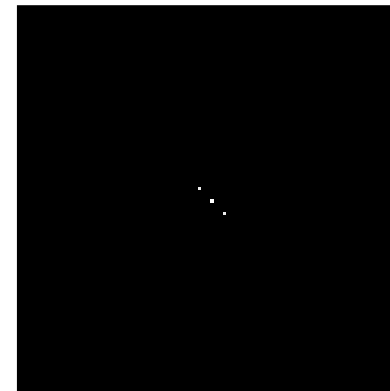
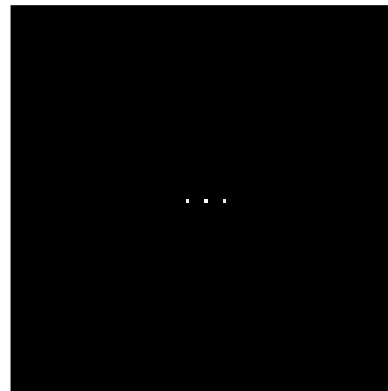
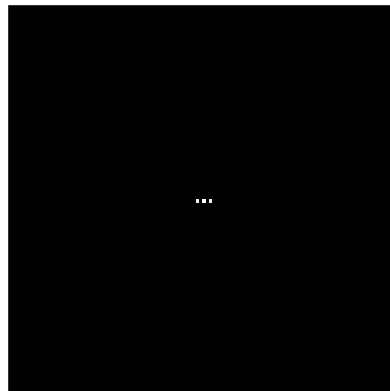
Frequency Analysis

Fourier analysis in images 图像的傅里叶变换

Spatial domain
images



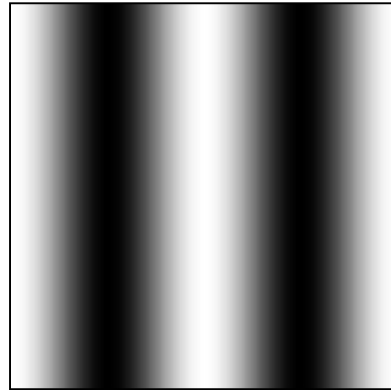
Fourier
decomposition
amplitude
images



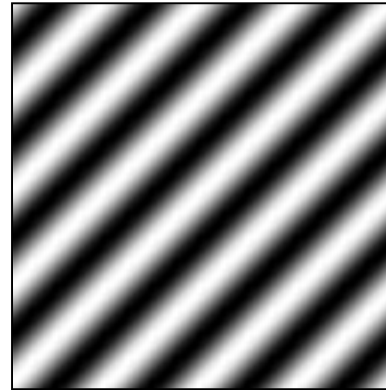
Frequency Analysis

Signals can be composed

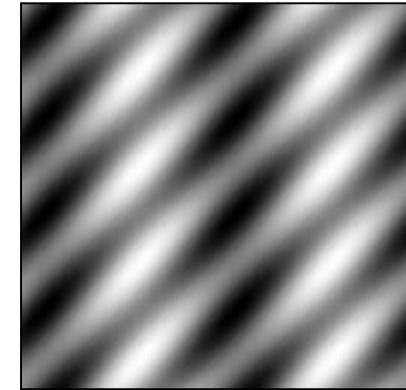
Spatial domain
images



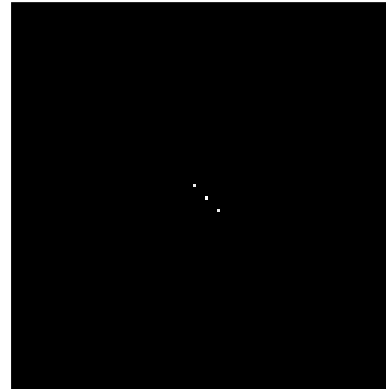
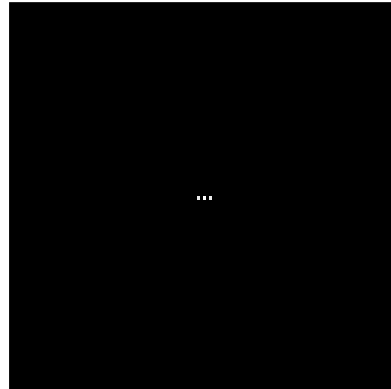
+



=

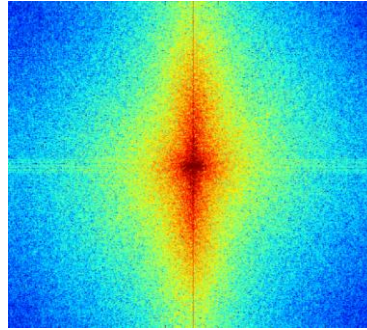
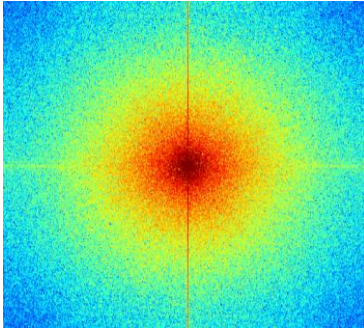


Fourier
decomposition
amplitude
images



Frequency Analysis

Spatial domain image and the Fourier amplitude image



Frequency Analysis



The Convolution Theorem 卷积定理

- The Fourier transform of the convolution of two functions is the product of their Fourier transforms

$$F[g * h] = F[g]F[h]$$

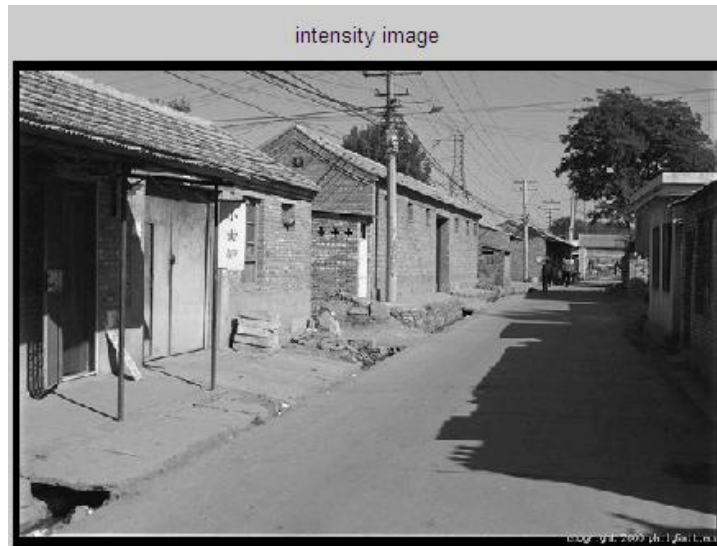
- **Convolution** in spatial domain is equivalent to **multiplication** in frequency domain!

$$g * h = F^{-1}[F[g]F[h]]$$

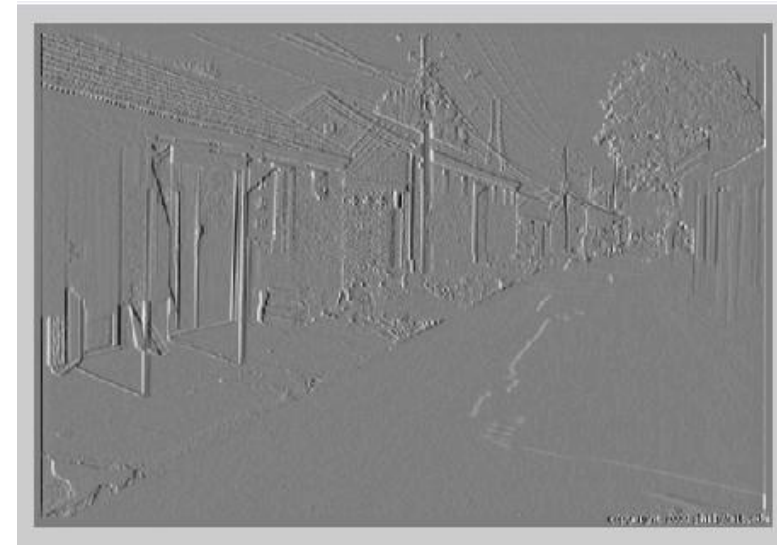
Frequency Analysis

Filtering in spatial domain

1	0	-1
2	0	-2
1	0	-1

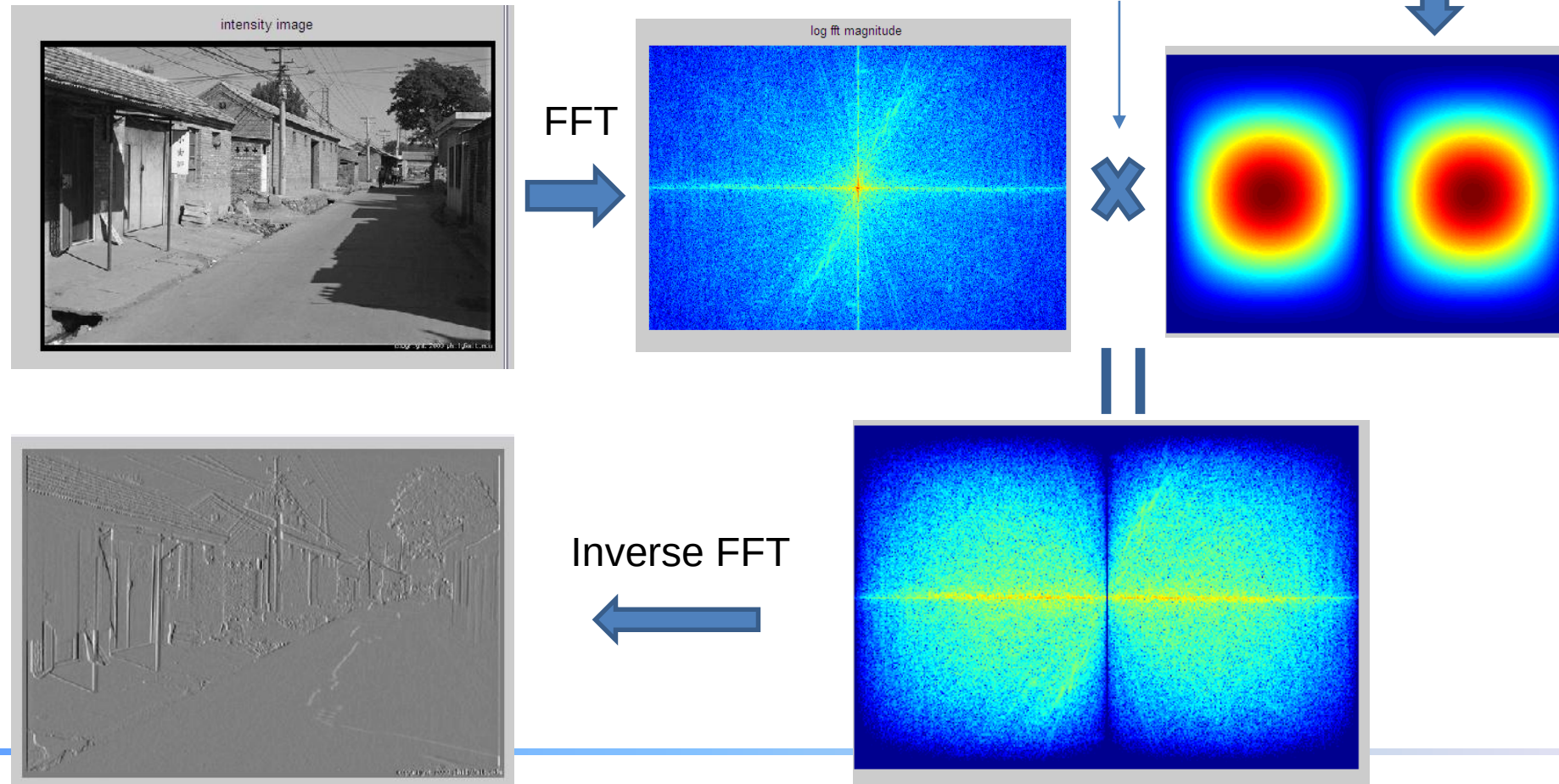


Convolution



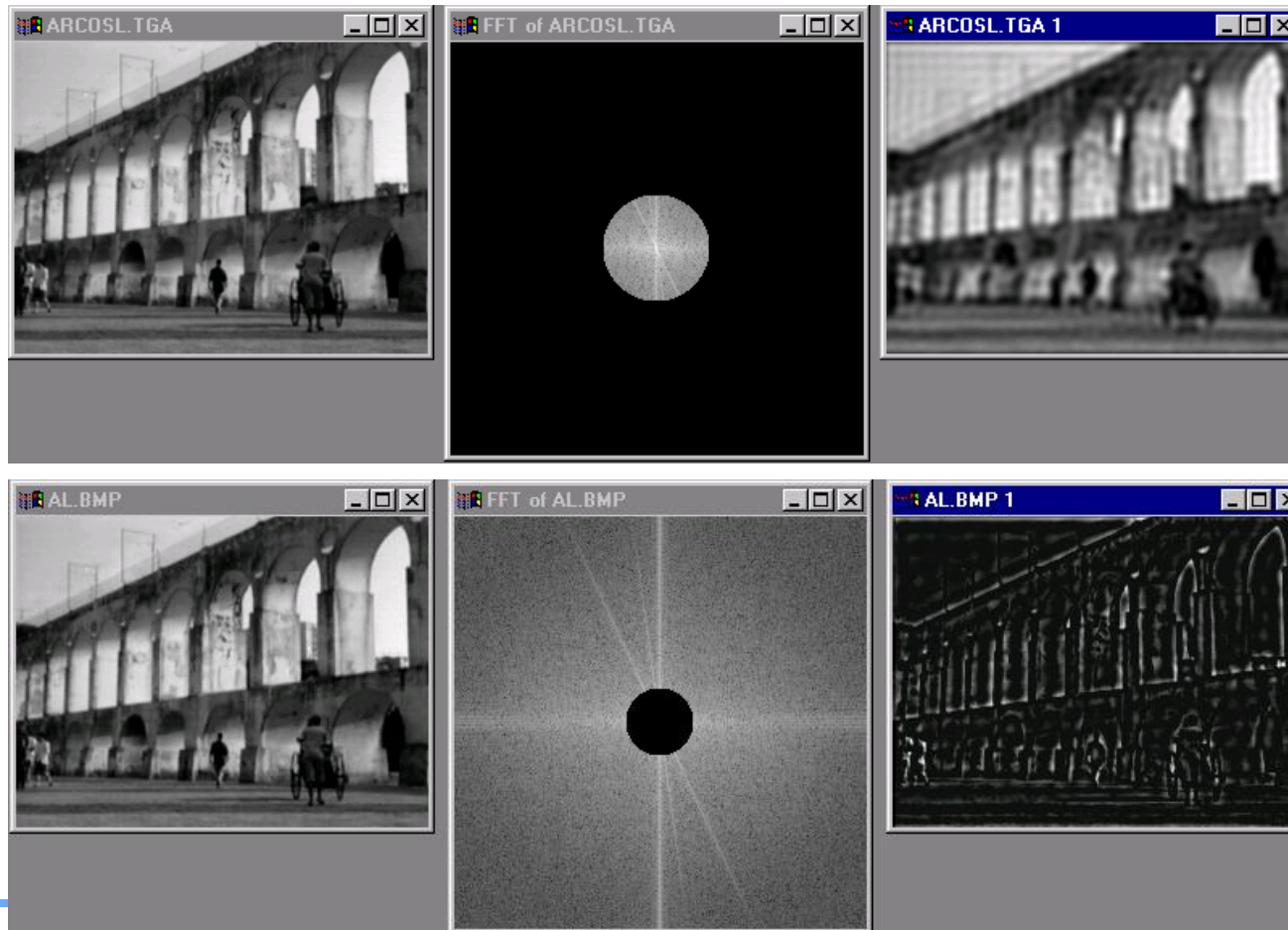
Frequency Analysis

Filtering in frequency domain



Frequency Analysis

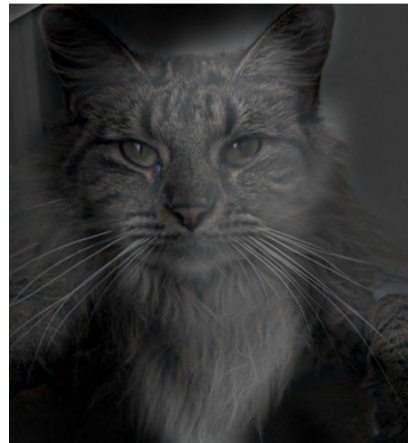
Low and High Pass filtering



Frequency Analysis

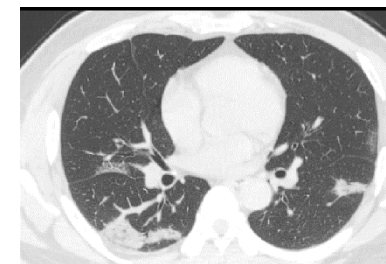
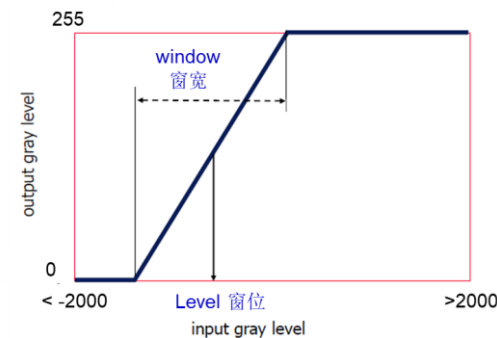
Hybrid image

Hybrid Image = Low-passed Image + High-passed Image



小结

- Point processing
 - Gamma correction
 - Histogram equalization
 - Window-center correction
- Filtering
 - Mean, gradient
 - Convolution, cross-correlation
 - Gaussian and median filters
- Frequency analysis
 - Sampling
 - Fourier analysis



$$h[m, n] = \sum_{k=-u}^u \sum_{l=-l}^l f[k, l] \cdot I[m + k, n + l]$$

$$F[g * h] = F[g]F[h]$$

